자유곡선을 이용한 효과적인 삼각형 메시 절단 방법

전체 한국어 번역 · 원문 대조본

Seung-Yong Lee · Seong-Hyeon Kweon · Seung-Hyun Yoon

Mathematics 2024, 12(10), 1432 DOI: 10.3390/math12101432

읽는 방법

원문 1-18쪽의 본문을 순서대로 번역했다. 각 한국어 번역 묶음 뒤에 대응하는 원문 페이지를 넣어 수식, 그림, 그래프, 표 및 참고문헌을 바로 대조할 수 있게 했다. 한국어 번역 분량에 따라 한 원문 페이지의 번역이 여러 쪽으로 이어질 수 있다.

본문·초록·그림 설명·표 항목·알고리즘·저자 정보·결론·지원 정보·출판사 면책문을 번역했다. 참고문헌은 제목을 한국어로 번역하고, 정확한 서지 정보는 대응 원문에 그대로 유지했다. 수식 (5)의 정확한 행렬 표기는 원문 페이지를 참조한다.

용어

원어	이 번역본의 표기
ruled surface	직선생성곡면(생성선으로 구성하는 곡면)
cross-section	단면 / 절단면
solid-type mesh	닫힌 입체를 나타내는 표면 메시
volumetric mesh	볼륨 메시(예: 사면체 요소 메시)
degenerate triangle	퇴화 삼각형
constrained Delaunay triangulation	제약 델로네 삼각분할

번역 안내: 학습용 비공식 AI 번역이다. 원문의 주장과 수치를 충실히 옮겼으며, 원문에 포함된 수학적 표현을 번역자가 교정한 것으로 해석하지 않아야 한다. 원문 저작권 © 2024 저자들, CC BY 4.0. 번역 제작: 2026-10-14.

논문 정보 · 초록 · 1. 서론

자유곡선을 이용한 효과적인 삼각형 메시 절단 방법

An Effective Method for Slicing Triangle Meshes Using a Freeform Curve

저자: Seung-Yong Lee, Seong-Hyeon Kweon, Seung-Hyun Yoon(교신저자)

소속: 대한민국 서울 04620, 동국대학교 멀티미디어공학과

연락처: randysylee@dongguk.edu (S.-Y.L.); kshyunee@dgu.ac.kr (S.-H.K.); shyun@dongguk.edu (교신저자)

게재: Mathematics 2024, 12, 1432. DOI: 10.3390/math12101432

학술 편집자: Shiaofen Fang. 접수: 2024년 3월 13일. 수정: 5월 1일. 승인: 5월 5일. 출판: 5월 7일.

저작권: © 2024 저자들. 발행 허가자: MDPI, 스위스 바젤. 원문은 Creative Commons Attribution(CC BY) 4.0 조건으로 배포되는 오픈 액세스 논문이다.

초록

3D 폴리곤 메시의 절단은 가상 수술, 의복 시뮬레이션, 게임 개발 등 다양한 응용에서 기본적인 연산이다. 기존 방법은 주로 하나의 직선이나 매끄러운 곡선을 근사하는 선분 집합을 사용해 메시를 절단한다. 본 논문은 자유곡선을 선분으로 이산화하지 않고 삼각형 메시를 자유롭게 절단하는 새로운 방법을 소개한다. 사용자는 화면에 스트로크를 그려 원하는 절단 경로를 정의한다. 이후 이 스트로크를 근사하는 자유곡선을 생성하고 사용자의 시선 방향으로 확장하여 직선생성곡면(ruled surface)을 만든다.

직선생성곡면과 삼각형 메시 사이의 교차를 효율적으로 계산하기 위해 직선-곡면 교차(LSI) 문제를 평면-곡선 교차(PCI)와 직선-직선 교차(LLI)의 두 하위 문제로 분해한다. 교차점들을 연결하여 폴리라인을 만들고, 이를 이용해 메시를 여러 서브메시로 절단한다. 서브메시가 닫힌 입체를 이루도록 폴리라인을 따라 직선생성곡면을 트리밍하여 단면을 만들고 해당 서브메시와 병합한다. 제안 방법은 효율성과 정확도를 유지하면서 직선과 자유롭게 휘어진 경로를 모두 포함하는 임의의 경로를 따라 삼각형 메시를 절단할 수 있게 한다. 다양한 메시 절단 예시를 포함한 실험 결과로 제안 방법의 유효성을 보여준다.

키워드: 메시 처리; 메시 절단; 자유곡선; 직선생성곡면; 직선-곡면 교차

MSC: 68U05

1. 서론

폴리곤 메시는 단순하고 3차원 형상을 효과적으로 표현할 수 있기 때문에 컴퓨터 그래픽스와 기하 모델링에서 널리 사용된다. 지난 수십 년 동안 재메시, 평활화, 단순화, 변형 등을 포함한 다양한 폴리곤 메시 처리(PMP) 기법[1-4]이 개발되었다. 메시 절단은 메시를 여러 서브메시로 나누는 것을 뜻하며, 이를 위한 많은 알고리즘이 개발되었다. 그러나 대부분의 방법은 사용자가 지정한 직선에 의존하고 자유곡선 활용에는 상대적으로 관심이 적었다. 본 논문은 입체형 삼각형 메시를 자유곡선으로 절단하는 효과적인 방법을 제안하며, 절단 결과의 서브메시도 입체형이라고 가정한다.

하나의 직선으로 삼각형 메시를 절단하는 과정은 비교적 간단하다. 본질적으로 직선을 시선 방향으로 압출해 만든 절단 평면과 삼각형 에지 사이의 교차 문제로 환원된다(그림 1a,b). 삼각형 에지의 매개변수 직선식 $l(t) = (x(t), y(t), z(t))$을 평면의 음함수식 $ax + by + cz + d = 0$에 대입하면 교차점을 쉽게 계산할 수 있다. 이후 교차점을 연결한 폴리라인으로 메시를 절단하면 열린 경계, 즉 구멍을 갖는 여러 서브메시가 생긴다. 각 경계는 구멍 메우기나 단순 삼각분할로 만든 단면으로 채운다. 그림 1b는 직선으로 Bunny 모델을 절단해 두 개의 닫힌 서브메시를 만든 결과이다. 평면 기반 절단은 단순하지만 절단 평면의 경계가 무한히 확장되기 때문에 의도하지 않은 결과가 발생할 수 있다.

mathematics

MDPI

Article

An Effective Method for Slicing Triangle Meshes Using a Freeform Curve

Seung-Yong Lee, Seong-Hyeon Kweon and Seung-Hyun Yoon *

Department of Multimedia Engineering, Dongguk University, Seoul 04620, Republic of Korea; randysylee@dongguk.edu (S.-Y.L.); kshyunee@dgu.ac.kr (S.-H.K.)
* Correspondence: shyun@dongguk.edu

Abstract: Slicing 3D polygonal meshes is a fundamental operation in various applications such as virtual surgery, garment simulation, and game development. Existing methods primarily slice meshes using either a single line or a set of line segments approximating a smooth curve. This paper introduces a novel approach to freely slice a triangle mesh using a freeform curve without discretizing it into line segments. The user draws a stroke on the screen, defining the desired cutting trajectory. Subsequently, a freeform curve approximating this stroke is generated and extended into a ruled surface in the user's viewing direction. To efficiently compute intersections between the ruled surface and a triangle mesh, the Line–Surface Intersection (LSI) problem is broken down into two subproblems: Plane–Curve Intersection (PCI) followed by Line–Line Intersection (LLI). Intersection points are then connected to form polylines, effectively cutting the mesh into multiple submeshes. To ensure the solidity of the submeshes, cross-sections are generated by trimming the ruled surface along the polylines and merged with the corresponding submeshes. Our method empowers users to slice triangle meshes along arbitrary trajectories encompassing both straight and freely curved paths while preserving efficiency and accuracy. The effectiveness of the proposed approach is demonstrated through experimental results showing various examples of mesh slicing.

Keywords: mesh processing; mesh slicing; freeform curve; ruled surface; line–surface intersection

MSC: 68U05

check for updates

Citation: Lee, S.-Y.; Kweon, S.-H.; Yoon, S.-H. An Effective Method for Slicing Triangle Meshes Using a Freeform Curve. *Mathematics* **2024**, *12*, 1432. https://doi.org/10.3390/math12101432

Academic Editor: Shiaofen Fang

Received: 13 March 2024
Revised: 1 May 2024
Accepted: 5 May 2024
Published: 7 May 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Polygonal meshes are widely used in computer graphics and geometric modeling for their simplicity and effectiveness in representing three-dimensional shapes. Over the past few decades, various techniques [1–4] have been developed for polygon mesh processing (PMP), including remeshing, smoothing, decimation, deformation, and more. Mesh slicing refers to dividing a mesh into multiple submeshes, and numerous algorithms have been developed for this purpose; however, most methods rely on user-specified lines for slicing polygonal meshes, with less focus on using freeform curves. In this paper, we propose an effective method for slicing solid-type triangle meshes using a freeform curve, assuming that the resulting sliced submeshes are also solid-type.

Slicing a triangle mesh with a single line is a relatively straightforward process, essentially reduced to solving the intersection problem between triangle edges and the slicing plane formed by extruding the line in the viewing direction (refer to Figure 1a,b). By substituting the parametric line equations $l(t) = (x(t), y(t), z(t))$ of triangle edges into the implicit plane equation $ax + by + cz + d = 0$, the intersection points can be easily computed. Subsequently, the triangle mesh is cut by the polylines connecting these intersection points, resulting in multiple submeshes with open boundaries (holes). Each boundary is then filled with the cross-section formed by hole-filling or a simple triangulation method. Figure 1b shows the result of slicing the Bunny model using a line, with both resulting submeshes being solid. While plane-based mesh slicing is straightforward and simple, it can lead to

1. 서론 (계속) · 식 (1)

이 한계를 해결하기 위해 Lee 등[5]은 유한한 평면으로 원하는 부분만 선택적으로 절단하는 방법을 제안했다. 그러나 유한한 평면을 사용해도 복잡한 형태의 서브메시로 나누려면 사용자가 상당한 조작을 해야 한다.

그림 1. 메시 절단 유형: (a) 직선, (b) (a)에 의한 절단 결과, (c) 자유곡선, (d) (c)에 의한 절단 결과.

자유곡선을 이용한 메시 절단은 더 큰 자유도와 가능성을 제공한다. 그러나 기존 방법[6-8]은 그림 2b처럼 자유곡선을 선분 집합으로 이산화한 뒤 직선 기반 방법을 적용하는 단순한 접근을 취한다. 이 방식은 필연적으로 근사 오차를 발생시키며, 그림 2d처럼 불필요한 경계 에지를 지나치게 많이 생성한다. 본 논문에서는 자유곡선을 선분으로 이산화하지 않는 새로운 메시 절단 방법을 제안한다. 자유곡선과 삼각형 사이의 교차점을 직접 계산하고(그림 2a), 그 교차점을 연결하여 삼각형을 절단함으로써 최적의 경계 에지를 생성한다(그림 2c).

그림 2. 자유곡선을 이용한 메시 절단: (a) 삼각형과 자유곡선의 교차점, (b) 자유곡선의 선분 이산화, (c) 교차점에 의한 절단 결과, (d) 이산 선분에 의한 절단 결과.

제안 방법은 2차원 곡선 C(u)에서 시작하여 사용자의 시선 방향 r로 선형 압출함으로써 직선생성곡면 R(u,v) = C(u) + vr로 확장한다(그림 1c,d). 이후 삼각형 에지와 이 곡면 사이의 교차점을 계산하며, 이는 직선-곡면 교차(LSI) 문제로 환원된다. 일반적으로 매개변수 곡면 $S(u,v) = (x_1(u,v), y_1(u,v), z_1(u,v))$와 직선 $l(t) = (x_2(t), y_2(t), z_2(t))$의 LSI 문제는 식 (1)처럼 표현된다.

식 (1): $x_1(u,v) = x_2(t)$, $y_1(u,v) = y_2(t)$, $z_1(u,v) = z_2(t)$.

수식의 벡터·첨자·정확한 조판은 뒤에 붙인 원문 2쪽에서 확인할 수 있다.

undesired results due to the infinite extension of the slicing plane boundary. To address this limitation, Lee et al. [5] proposed a method using a bounded plane to selectively slice only the desired portion. However, even when using a bounded plane, slicing into submeshes with complex shapes requires significant user intervention.

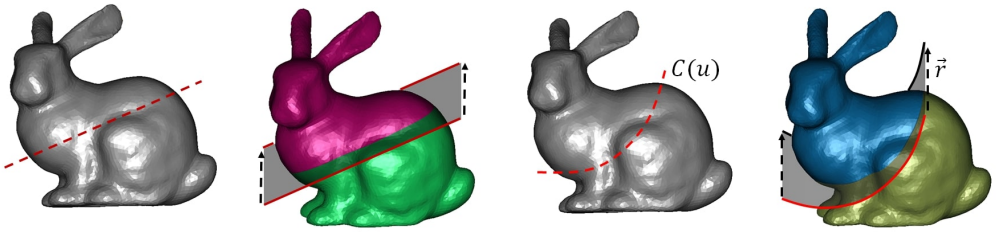


Figure 1. Mesh slicing types: (**a**) a straight line, (**b**) slicing result by (**a**), (**c**) a freeform curve, (**d**) slicing result by (**c**).

Mesh slicing with a freeform curve offers users greater flexibility and potential; however, existing methods [6–8] take a simple approach to discretizing a freeform curve into a set of line segments and applying a line-based method, as shown in Figure 2b. This approach inevitably leads to approximation errors and generates an excessive number of unnecessary boundary edges, as shown in Figure 2d. In this paper, we propose a new method for mesh slicing using a freeform curve without discretizing it into line segments. We directly compute the intersection points between a freeform curve and triangles (see Figure 2a) and cut the triangles by connecting these intersection points, resulting in optimal boundary edges (see Figure 2c).

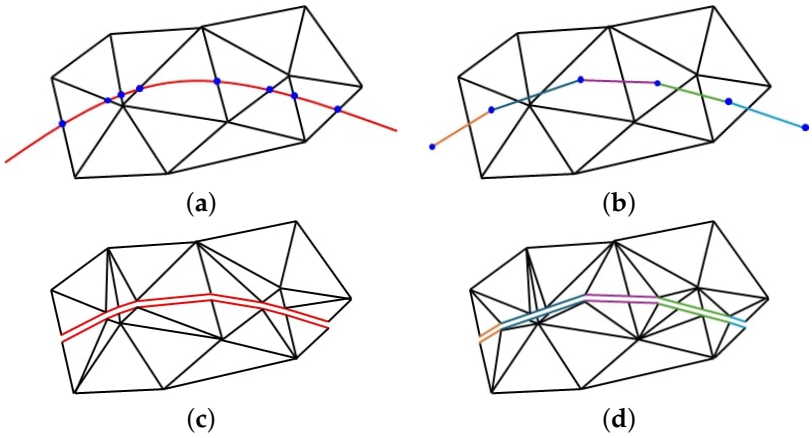


Figure 2. Mesh cutting with a freeform curve: (**a**) intersection points between triangles and a freeform curve, (**b**) discretizing a freeform curve into line segments, (**c**) result of cutting by intersection points, and (**d**) result of cutting by discrete line segments.

Our method starts with a two-dimensional curve $C(u)$ and extends it to a ruled surface $R(u,v) = C(u) + v\vec{r}$ by linearly extruding it in the user's viewing direction $\vec{r}$ (see Figure 1c,d). Then, we compute the intersection points between edges of triangles and the ruled surface, which is reduced to a Line–Surface Intersection (LSI) problem. In general, for a parametric surface $S(u,v) = (x_1(u,v), y_1(u,v), z_1(u,v))$ and a line $l(t) = (x_2(t), y_2(t), z_2(t))$, the LSI problem can be formulated as follows:

$$\begin{cases} x_1(u,v) &= x_2(t) \\ y_1(u,v) &= y_2(t) \\ z_1(u,v) &= z_2(t) \end{cases}. \tag{1}$$

1. 서론 (계속) · 식 (2) · 연구의 장점

직선 l(t)의 선형 변수 t를 소거하면 미지수 u와 v에 대한 비선형 연립방정식, 즉 f(u,v) = 0 및 g(u,v) = 0을 얻는다[식 (2)]. 매개변수 영역을 반복적으로 세분화하여 초기 해 (u_0,v_0)를 추정한 뒤 Newton-Raphson 방법[9]을 적용하면 이 연립방정식을 효율적으로 풀 수 있다. 그러나 많은 직선, 즉 모든 삼각형의 에지를 처리할 때는 상당한 계산 비용이 발생할 수 있다.

본 논문은 LSI 문제를 직선생성곡면 R(u,v)로 한정하고 직선 l(t)와의 교차점을 계산하는 효율적인 알고리즘을 제안한다. 제안 방법은 LSI를 평면-곡선 교차(PCI), 그다음 직선-직선 교차(LLI)의 두 하위 문제로 분해한다. 먼저 PCI를 풀어 중간 교차점 C(u*)를 구하고, 이후 LLI를 풀어 최종 교차점 R(u*,v*)를 계산한다. 직선 절단과 마찬가지로 교차점을 연결한 폴리라인으로 삼각형 메시를 절단한다.

곡선을 사용할 때 단면은 더 이상 평면이 아니며 직선생성곡면 R(u,v)의 부분 영역이어야 한다. 모든 교차 매개변수가 R(u,v)의 매개변수 영역에서 계산되므로, 곡면을 잘라 단면을 생성하는 과정은 비교적 간단하다. 그림 1d는 자유곡선으로 Bunny 모델을 절단한 결과이며, 그림 3은 전체 처리 과정을 보여준다.

그림 3. 제안 방법의 개요.

제안 방법의 장점은 다음과 같다.

- 유효성: 사용자가 제공하는 직선 스트로크와 매우 복잡한 스트로크 모두에 대해 메시를 여러 서브메시로 효과적으로 절단한다.
- 유연성: 직선생성곡면의 샘플 점 수를 조절하여 생성되는 단면의 해상도를 쉽게 제어한다.
- 효율성과 강건성: LSI를 PCI와 LLI로 분해하여 교차점 계산 비용을 줄이는 동시에 제안 방법의 강건성을 높인다.

논문의 구성은 다음과 같다. 2절은 표면 메시 절단 관련 연구를 간략히 검토한다. 3절은 자유곡선을 이용한 메시 절단 알고리즘을 설명한다. 4절은 다양한 실험 결과와 응용 가능성을 제시한다. 5절에서는 결과를 논의하고 결론 및 향후 연구를 제시한다.

By eliminating the linear variable t of $l(t)$, we can obtain a system of nonlinear equations with two unknowns u and v, as follows:

$$\begin{cases} f(u,v) &= 0 \\ g(u,v) &= 0 \end{cases}. \tag{2}$$

This system can be efficiently solved by guessing an initial solution (u_0, v_0) through iterative subdivision of the parametric domain, then applying the Newton–Raphson method [9]. However, this approach may incur significant computational costs when dealing with a large number of lines, i.e., edges of all triangles.

In this paper, we confine the LSI problem to a ruled surface $R(u,v)$ and propose an efficient algorithm for calculating the intersection points with a line $l(t)$. Our method decomposes the LSI problem of a ruled surface into two subproblems: Plane–Curve Intersection (PCI) followed by Line–Line Intersection (LLI). First, an intermediate intersection point $C(u^*)$ is determined by solving the PCI problem. Subsequently, the final intersection point $R(u^*, v^*)$ is computed by solving the LLI problem. Similar to the case with the line, the triangle mesh is cut by the polylines connecting intersection points. When using a curve, the cross-section is not planar anymore, and should be a subregion of the ruled surface $R(u,v)$. As all intersection parameters are computed within the parameter domain of $R(u,v)$, it is relatively straightforward to generate the cross-section by cutting the ruled surface. Figure 1d shows the result of slicing the Bunny model using a freeform curve. The overall process of the proposed method is illustrated in Figure 3.

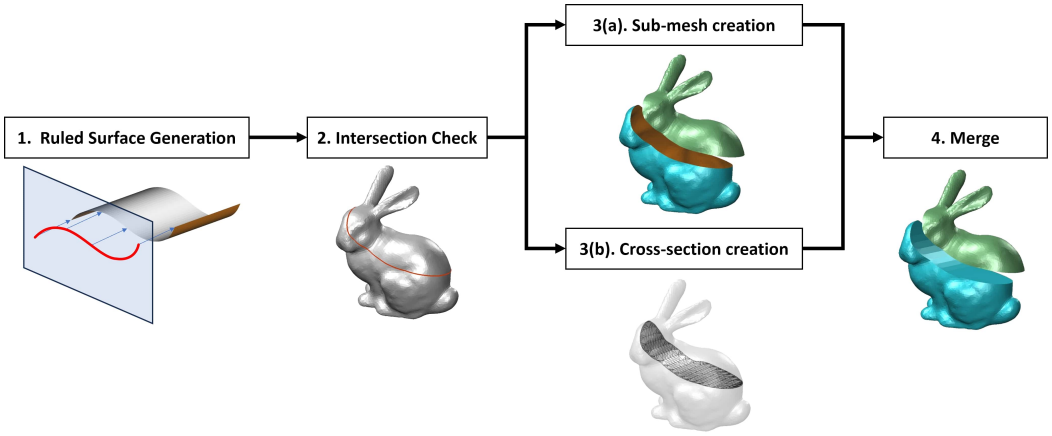


Figure 3. Overview of the proposed method.

Our method has several advantages, which can be summarized as follows:

- **Effectiveness.** The proposed method enables an effective slicing of a given triangle mesh into multiple submeshes both for straight strokes and for highly complex strokes provided by the user.
- **Flexibility.** Our method provides easy control over the resolution of the generated cross-sections by adjusting the number of sampling points on the ruled surface.
- **Efficiency and robustness.** The LSI problem is decomposed into two subproblems, first the PCI problem and then the LLI problem. This decomposition reduces the computational cost of calculating intersection points while simultaneously improving the robustness of the proposed method.

The remainder of this paper is organized as follows. In Section 2, we provide a brief review of previous works on surface mesh slicing. Section 3 describes the mesh slicing algorithm using a freeform curve. Various experimental results and potential applications are presented in Section 4, then we discuss the results, conclude the paper, and propose future work in Section 5.

2. 관련 연구

이 절에서는 본 논문의 기여와 직접 관련된 기존 연구, 특히 메시 절단 기법과 다양한 자유형 모델 사이의 교차 계산을 검토한다. 1절의 직선 기반 접근과 유사하게 대부분의 방법은 3D 물체를 자를 절단 평면을 만드는 단계를 포함한다. Chu 등[10]은 일반적인 3D 프린터의 출력 크기를 초과하는 대형 모델을 나누기 위한 절단 평면 생성 방법을 소개했다. 사용자가 지정한 직선으로 절단 평면을 정의하여 모델을 3D 프린팅용으로 분할한다.

Kang과 Kim[11]은 3D 의복 모델을 패치로 나누는 과정을 연구했다. 여러 선분으로 구성된 사용자 정의 절단선을 허용하여 하나의 직선만으로 절단하는 제약을 극복한다. 절단선이 한 선분이면 기존의 절단 평면을 생성하고, 복잡한 선이면 사용자가 선들을 연결하여 화면에 투영된 2D 폴리곤을 만든다. 이후 평면 또는 폴리곤과 삼각형의 교차로 패치를 정의한다. 이 접근은 메시의 앞면만 자르는 반면, 제안 방법은 메시를 완전히 절단하여 서브메시로 나눈다.

Owada 등[12]은 사용자 스트로크를 근사하는 직선들로 변환한 뒤 시선 방향으로 압출하여 절단 평면을 만드는 방법을 제시했다. 이들 방법은 스트로크를 일련의 선분으로 근사하므로 절단 자유도와 정확도가 낮아지는 공통적인 한계가 있다. 이를 극복하기 위해 Mitani[6]는 화면에 자유롭게 그린 스트로크로 절단 경로를 지정하는 알고리즘을 제안했다. 폴리곤 메시를 화면에 투영하여 2D 교차 검사를 수행하며, 노드 스냅, 정점을 직선으로 이동시키기, 경로 단순화 등을 사용해 정확도보다 단순성과 강건성을 우선한다. 사용하기 쉽고 계산 오류를 일으킬 수 있는 퇴화 삼각형을 방지하지만, 스트로크를 크게 수정하면 절단 경계의 정확도가 떨어질 수 있다.

Owada 등[8]은 내부 텍스처를 드러내기 위해 폴리곤 메시로 시각화하는 볼륨 메시 절단 인터페이스를 제시했다. 제안 방법과 유사하게 자유 스트로크를 사용하지만, 이를 선분으로 샘플링하고 화면에 수직한 방향으로 스윕하여 휘어진 이산 평면을 생성한다. 이 평면으로 CSG 연산을 수행하여 볼륨 메시를 절단하고, 단면을 매개변수화하여 폴리곤 메시의 내부 텍스처를 시각화한다. 스트로크 자유도는 해결하지만 자유 스트로크의 이산화 문제는 남는다.

여러 연구[13-15]는 메시 조작을 위한 가상 절단 도구를 다뤘다. Khan 등[13]은 가상 메스의 날이 정의하는 절단 평면과 교차하는 면을 제거하고, 교차점을 남은 면의 경계에 연결해 경계를 재생성한다. 단면은 초기 삼각분할을 여러 수준으로 세분화하여 생성한다. Kamarianakis 등[14]은 가상 메스와 상호작용할 때 발생할 수 있는 메시 세분화 문제를 다뤘다. 사면체와의 교차가 감지되면 교차 정점을 메스 경로에 투영해 불필요한 세분화를 방지하고, 투영된 경로를 따라 메시를 분리한다. Caligiana 등[15]은 Phantom 햅틱 장치로 가상 수술 도구를 조작하고, 교차 삼각형을 최장 에지 세분화 전략으로 나눈다. 이러한 접근은 실시간 상호작용을 제공하고 특정 응용에 적합할 수 있지만, 추가 하드웨어가 필요하거나 도구 조작이 복잡해질 수 있다. 이에 비해 제안하는 스트로크 기반 접근은 효율적인 메시 절단을 유지하면서 사용자 직관과 사용 편의성을 우선한다.

메시 절단 알고리즘은 교차 유형에 따라 이산-이산(평면-평면, 평면-직선, 직선-직선), 이산-자유형(평면-곡면, 평면-곡선, 직선-곡면, 직선-곡선), 자유형-자유형(곡면-곡면, 곡선-곡면, 곡선-곡선)으로 분류된다. 제안 방법은 삼각형과 직선 생성곡면이 교차하는 이산-자유형 사례이다.

2. Related Work

In this section, we examine previous research directly related to the contributions of this paper, particularly including mesh slicing techniques and the computation of intersections among diverse freeform models. Similar to the line-based approach mentioned in Section 1, most methods involve the step of creating a cutting plane to slice a 3D object. Chu et al. [10] introduced a method for creating a cutting plane to slice large models that exceed the general 3D printer's printable size. They used a user-specified line to define the cutting plane, allowing the user to partition the model for 3D printing. Kang and Kim [11] explored the process of cutting a 3D garment model into patches. Their method overcomes the limitations of slicing using only a line by allowing for user-defined cutting lines composed of multiple line segments. If the cutting line is a single segment, a traditional slicing plane is generated; however, for complex lines, users must connect them, creating a projected 2D polygon on the screen. Subsequently, triangles are intersected with either a plane or a polygon to define the patches. This approach is limited to cutting only the front side of the mesh, whereas our method supports completely cutting the mesh into submeshes. Owada et al. [12] presented a method where user strokes are converted into straight lines approximating them. A slicing plane is then created by extruding these lines in the viewing direction. These methods have the common limitation of approximating the user's strokes using a series of line segments, resulting in reduced slicing freedom and lower precision. To overcome this limitation, Mitani [6] proposed a slicing algorithm in which the user defines a cutting path with freehand strokes on the screen. This method performs intersection checks in 2D by projecting the polygonal mesh onto the screen, and prioritizes simplicity and robustness over precision by employing techniques such as node-snapping, vertex-to-line movement, and path simplification. While this approach offers ease of use and prevents degenerate triangles that could cause computational errors, the significant modifications to the stroke can compromise the precision of the slicing boundary.

Owada et al. [8] presented a slicing algorithm similar to our approach, featuring an interactive interface for volumetric mesh slicing visualized with a polygonal mesh to reveal the inner texture. A user-drawn freeform stroke is sampled into line segments and swept in a direction orthogonal to the screen, generating a curved discrete plane that slices the volumetric mesh through a constructive solid geometry (CSG) operation. This method then parameterizes the cross-section to visualize the inner texture of the polygonal mesh. Although this addresses the freedom of stroke issue, the issue of discretizing the freeform stroke remains.

Several methods [13–15] have explored virtual cutting tools for mesh manipulation. Khan et al. [13] presented a method for cutting polygonal meshes. Their approach removes faces that intersect with the slicing plane defined by a virtual scalpel blade. Subsequently, the boundary is regenerated by connecting the intersection points to the boundaries of the remaining faces. For generating a cross-section, they refined an initial triangulation through a multi-level refinement process. Kamarianakis et al. [14] addressed potential mesh subdivision issues during virtual scalpel interactions. When an intersection with a tetrahedral polygon is detected, the intersecting vertex is projected onto the scalpel's path to prevent unnecessary subdivision. The mesh is subsequently separated along this projected path. Caligiana et al. [15] employed a haptic device (Phantom) to facilitate interaction with a virtual surgical tool for mesh slicing, where intersecting triangles are subdivided using a longest-edge refinement strategy. While these virtual tool approaches offer real-time interaction and may be well-suited for specific applications, they could require additional hardware or introduce complexities in tool manipulation. In contrast, our stroke-based approach prioritizes user intuition and ease of use while maintaining efficient mesh slicing functionality.

Mesh slicing algorithms can be categorized based on the intersection types involved: discrete-discrete (plane–plane, plane–line, line–line), discrete-freeform (plane–surface, plane–curve, line–surface, line–curve), and freeform-freeform (surface–surface, curve–surface, curve–curve). Our method can be classified as a discrete-freeform case, wherein

2. 관련 연구 (계속) · 3.1 곡면 생성 · 3.2 절단

Lu 등[16]은 곡면을 정사각형 패치로 이산화하고, 각 패치 모서리 정점의 매개변수를 평면 음함수식에 대입해 부호를 분석했다. 에지 양 끝의 부호가 다르면 에지 위에 교차점이 있음을 뜻한다. Sharma 등[17]은 삼각형 영역을 중심으로 매개변수 Bézier 삼각형 곡면과 평면의 교차를 찾는 추적 방법을 제안했다. Bézier 곡면의 매개변수식과 평면 방정식으로 교차점을 계산하며, 초기 교차점에서 시작해 경계 또는 사전 정의된 종료 조건에 도달할 때까지 교차 곡선을 반복적으로 따라간다.

Li 등[18]은 본 연구와 유사하게 직선생성곡면과 평면을 교차시키는 방법을 소개했다. 준선(directrix)을 샘플링하고 생성선 방향의 선분을 만들어 평면과의 교차를 찾지만, 제안 방법과 달리 곡면을 이산화한다는 단점이 있다. Beer 등[19]은 직선 위의 주어진 점에 가장 가까운 곡면 점을 찾는 반복적 LSI 접근을 탐구하고 대안도 제시했다. Martin 등[20]은 LSI를 광선 추적 문제로 표현했다. 광선 방향에 수직인 법선을 가지며 광선의 원점을 지나는 두 평면의 교차선으로 광선을 간주하고, 곡면 점을 두 평면의 음함수식에 대입한 연립방정식을 Newton 방법으로 푼다. Busé 등[21]은 Bézier 패치의 행렬 기반 음함수 표현(MRep)으로 LSI를 해결했으며, Shen 등[22]은 일반 MRep 방법의 퇴화 사례를 처리하여 정확도를 높였다.

3. 자유곡선을 이용한 메시 절단

3.1. 직선생성곡면 생성

이 절에서는 화면상의 사용자 자유 스트로크로부터 직선생성곡면을 만드는 과정을 설명한다. 이 곡면은 특별한 종류의 자유곡면으로, 평면적 성질(가우스 곡률 0)을 가지므로 휘어진 평면과 유사하게 볼 수 있다. 그림 4는 화면에 그린 사용자 스트로크를 검은색으로 나타낸다. 스트로크를 따라 일정 간격으로 점을 샘플링한다. 각 샘플 점 s_i에서 시선 방향으로 광선 r(t)를 쏘아 삼각형 메시의 경계 평면 π_1, π_2와의 교차점 p_i, q_i를 계산한다. π_1 위의 교차점 p_i를 오차 한계 ε 안에서 근사하는 3차 B-spline 곡선 C(u)를 구성한다. 이후 매개변수 곡면을 식 (3)으로 정의한다.

식 (3): $R(u,v) = C(u) + vr,\ 0 \le u,v \le 1.$

여기서 $r = q_i - p_i$는 R(u,v)의 생성선에 대한 일정한 방향 벡터이다. 그림 4의 빨간색은 생성된 곡면이다. 이 곡면은 평면적 성질을 가지지만 평면처럼 음함수로 표현할 수 없으므로, 평면 절단보다 더 많은 계산 단계가 필요하다. 제안 기법은 사용자의 스트로크가 메시를 완전히 가로지르도록 요구하며 자유곡선과 기존 직선을 모두 지원한다.

3.2. 직선생성곡면에 의한 절단

곡면 R(u,v)를 생성한 뒤에는 곡면과 메시 삼각형 사이의 교차선을 계산한다. 이 과정을 ruled slicing이라고 부른다. 메시 삼각형 중 일부만 곡면과 교차하므로, 후보 삼각형을 찾는 것이 계산 효율 향상에 중요하다. 이를 위해 곡면을 삼각형으로 테셀레이션하고 그 경계 볼륨 계층(BVH)을 구성한 뒤, 메시 삼각형과 BVH의 교차 검사를 수행한다. 본 연구에서는 단순한 축 정렬 경계 상자(AABB)를 사용한다.

a triangle intersects with a ruled surface. Lu et al. [16] discretized the surface into square patches and analyzed the signs of each patch's corner vertices by substituting the parameters into the implicit plane equation. If opposite signs are detected on an edge, this indicates that an intersection point lies on that edge. Sharma et al. [17] proposed a tracing method for finding the intersection between a parametric Bézier triangular surface and a plane, specifically focusing on triangular domains. Their method utilizes the parametric equations of the Bézier surface and the plane equation to efficiently compute their intersection points. The tracing method starts from an initial intersection point and iteratively follows the intersection curve until it reaches a boundary or meets a predefined stopping condition.

Li et al. [18] introduced a method for intersecting a ruled surface with a plane, similar to our approach. Their method involves discretizing the ruled surface by sampling the directrix curve and generating line segments along the generator direction to find intersections with the plane; however, unlike our approach, this suffers from the drawback of discretizing the ruled surface. Beer et al. [19] explored iterative approaches to solving the Line-Surface Intersection (LSI) problem, which entails finding the closest point on the surface to a given point on the line, then presented alternative solutions to the LSI. Martin et al. [20] proposed a numerical approach to the LSI problem, framing it as a ray tracing problem. A ray is considered as the intersection line between two planes with normals perpendicular to the ray's direction and passing through the ray's origin. By substituting the surface point into the implicit equations of these two planes, the system of equations is solved using Newton's method. Busé et al. [21] introduced a matrix-based implicit representation for Bézier patches (MRep) to solve the LSI problem, while Shen et al. [22] tackled degenerate cases in the general MRep method, resulting in higher accuracy.

3. Mesh Slicing Using a Freeform Curve

3.1. Ruled Surface Generation

This section outlines the process of generating a ruled surface from the user's freehand stroke on the screen. A ruled surface, which is a specific kind of freeform surface, is considered akin to a curved plane due to its planarity (having zero Gaussian curvature). Figure 4 illustrates an example of a user's freehand stroke (depicted in black) on the screen. Points are sampled at regular intervals along the stroke. For each sampled point $\mathbf{s}_i$, a ray $r(t)$ is cast in the viewing direction to calculate the intersection points $\mathbf{p}_i$ and $\mathbf{q}_i$ with the bounding planes π_1 and π_2 of a triangle mesh, as demonstrated in Figure 4. A cubic B-spline curve $C(u)$ is then constructed to approximate the intersection points $\mathbf{p}_i$ on π_1 within an error bound ϵ. Subsequently, the parametric ruled surface is defined as

$$R(u, v) = C(u) + v\vec{r}, \quad \text{for } 0 \leq u, v \leq 1, \tag{3}$$

where $\vec{r} = \mathbf{q}_i - \mathbf{p}_i$ represents the constant directional vector of the generator of $R(u, v)$. Figure 4 shows the ruled surface (in red) generated using the proposed method. While a ruled surface exhibits planar characteristics, it cannot be represented by an implicit function such as a plane; therefore, slicing a mesh with a ruled surface requires more computational steps compared to using a plane. Our technique requires the user's stroke to intersect the mesh fully, accommodating both freeform curves and conventional straight lines.

3.2. Ruled Slicing

When the ruled surface $R(u, v)$ has been generated from the user's freehand stroke, the next step involves computing the intersection lines between the ruled surface and the triangles of a mesh to slice the mesh effectively. We refer to this process as *ruled slicing*. Because only a subset of the mesh's triangles intersect with the ruled surface, identifying these candidate triangles is crucial for improving computational efficiency. To facilitate this, we tessellate the ruled surface into triangles and construct its Bounding Volume Hierarchy (BVH), then carry out intersection tests between the mesh triangles and the constructed BVH. For our study, we employed a straightforward Axis-Aligned Bounding Box (AABB)

3.2 절단 (계속) · PCI 및 LLI · 식 (4)

그림 5는 R(u,v)의 BVH로 찾은 Bunny 모델의 교차 후보 삼각형을 노란색으로 나타낸다.

그림 4. 직선생성곡면 생성.

그림 5. 후보 면 검출(노란색): (a) 후보 면을 표시한 Bunny 모델의 앞면, (b) 뒷면.

$T = \{\Delta_1, \Delta_2, \cdots, \Delta_n\}$을 후보 삼각형 집합이라 하자. 각 삼각형 Δ_i의 정확한 교차 선분을 계산해야 한다. 삼각형의 세 에지에 대한 매개변수식을 $l_j(t)$, $j = 1,2,3$으로 나타내고, 각 에지와 R(u,v)의 교차점을 구한다. 이는 LSI 문제이며, 효율적인 해결 방법들이 연구되었다[20,21,23]. 본 논문에서는 그림 6a처럼 직선생성곡면과 직선의 LSI에 집중한다. 이를 두 가지 더 단순한 문제로 분해한다.

1. PCI 문제(그림 6b)

- l(t)와 생성선 방향 벡터 r를 포함하는 평면 π를 구성한다.
- 곡선 C(u)와 평면 π의 교차점 C(u*)를 구한다.

2. LLI 문제(그림 6c)

- C(u*)에서 시작해 r 방향으로 뻗는 생성선 m(v)를 만든다.
- m(v)와 l(t)의 교차점 R(u*,v*)를 계산한다.

그림 6에서 $l(t) = p + td$를 삼각형 에지의 선분이라고 하자. p를 지나고 법선 $n = d \times r$를 갖는 평면 π를 C(u)와 교차시킨다. 해당 교차점 C(u*)는 식 (4)를 풀어 얻는다.

식 (4): $\langle C(u) - p, n \rangle = 0$.

벡터 기호와 수식의 정확한 조판은 다음 원문 페이지에 보존되어 있다.

as the BVH type. Figure 5 illustrates the triangles (highlighted in yellow) of the Bunny model that intersect with the ruled surface, identified using the BVH of $R(u, v)$.

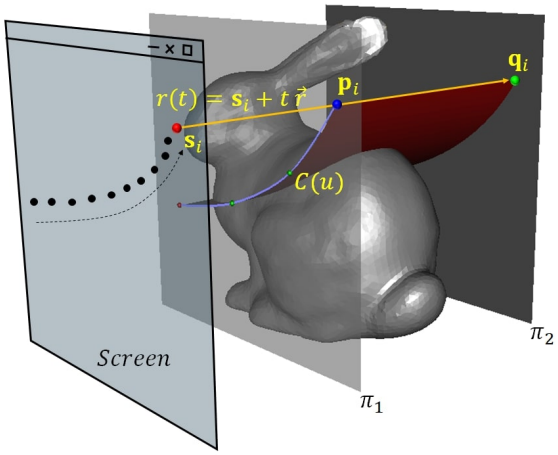
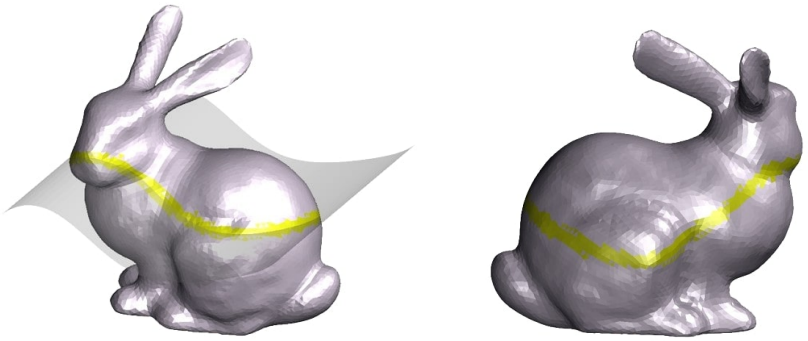


Figure 4. Generating a ruled surface.



(**a**) (**b**)

Figure 5. Detection of candidate faces (highlighted in yellow): (**a**) front view of Bunny model showing candidate faces and (**b**) back view of Bunny model showing candidate faces.

Let $T = \{\Delta_1, \Delta_2, \cdots, \Delta_n\}$ represent a set of candidate triangles. For each triangle Δ_i, we need to compute the exact intersection segment. We denote $l_j(t)$, where $j = 1, 2, 3$, as the parametric equations for the edges of triangle Δ_i. We then calculate the intersection points of $l_j(t)$ with $R(u, v)$. This task is commonly referred to as a Line–Surface Intersection (LSI) problem, for which various methods [20,21,23] have been devised to address it efficiently. In this paper, we focus on the LSI problem of a ruled surface with a line, as depicted in Figure 6a. Our LSI problem is decomposed into two simpler subproblems: (1) Plane–Curve Intersection (PCI) and (2) Line–Line Intersection (LLI), as follows:

1. PCI problem (Figure 6b)
   - Construct a plane π that contains $l(t)$ and the generator vector $\vec{r}$.
   - Determine the intersection point $C(u^*)$ between $C(u)$ and π.
2. LLI problem (Figure 6c)
   - Generate the ruling line $m(v)$ originating from $C(u^*)$ in the direction of $\vec{r}$.
   - Calculate the intersection point $R(u^*, v^*)$ between ruling line $m(v)$ and line $l(t)$.

In Figure 6, let $l(t) = \mathbf{p} + t\vec{d}$ denote the line segment of a triangle edge. A plane π passing through $\mathbf{p}$ with normal vector $\vec{n} = \vec{d} \times \vec{r}$ is intersected with $C(u)$ as shown in Figure 6b. The corresponding intersection point $C(u^*)$ can be found by solving the following equation:

$$< C(u) - \mathbf{p}, \vec{n} >= 0, \tag{4}$$

3.2 절단 (계속) · 식 (5) · 알고리즘 1

식 (4)의 ⟨·,·⟩는 내적을 뜻한다. 이 식은 곡선 C(u)와 평면 π 위에 동시에 놓이는 점을 찾는다. 미지수가 u 하나뿐이므로 식 (2)의 일반적인 LSI보다 교차점을 효율적으로 계산할 수 있다. 교차점 C(u*)에서 직선 m(v) = C(u*) + vr를 만든다. 이 직선은 r 방향을 가지므로 평면 π 위에도 놓인다. 따라서 최종 교차점 R(u*,v*)는 l(t)와 m(v)의 단순한 LLI 문제를 풀어 얻을 수 있으며, 식 (5)의 선형시스템으로 표현된다.

식 (5)의 행렬·벡터 표기는 다음 원문 7쪽에 그대로 보존했다.

결과적으로 제안 방법은 기하학적 해석을 바탕으로 두 미지수 u,v를 갖는 식 (2)를 더 단순한 식 (4)와 식 (5)로 분해한다. 이 식들을 순차적으로 풀어 교차점을 효율적이고 강건하게 계산한다. 알고리즘 1은 제안 방법으로 삼각형 메시와 곡면의 교차선을 계산하는 방법을 보여준다.

그림 6. (a)의 LSI를 (b) 평면-곡선 교차(PCI), (c) 직선-직선 교차(LLI)로 분해.

알고리즘 1. 후보 삼각형과 직선생성곡면의 교차

입력: 후보 삼각형 집합 $T = \{\Delta_i\}$, 직선생성곡면 $R(u,v) = C(u) + vr$.

1. T의 각 삼각형 Δ_i에 대해 반복한다.

2. Δ_i의 각 에지 e_j에 대해 반복한다.

3. e_j의 매개변수 직선 $l_j(t)$를 구성한다.

4. $l_j(t)$와 생성선 방향 벡터 r를 포함하는 평면 π를 만든다.

5. π와 C(u)의 교차점 C(u*)를 계산한다.

6. C(u*)에서 시작하는 매개변수 직선 m(v)를 만든다.

7. $l_j(t)$와 m(v)의 교차점 R(u*,v*)를 계산한다.

8. 에지 반복을 종료한다.

9. 삼각형 반복을 종료한다.

where $< \cdot, \cdot >$ represents an inner product. Equation (4) identifies the point lying simultaneously on the curve $C(u)$ and the plane π. Because Equation (4) involves only one unknown u, the corresponding intersection point can be computed more efficiently than the general LSI problem in Equation (2). Starting from the intersection point $C(u^*)$, we create a line $m(v) = C(u^*) + v\vec{r}$. As $m(v)$ has the direction of $\vec{r}$, it also lies on the plane π; thus, the final intersection point $R(u^*, v^*)$ can be found by solving a simple LLI problem between $l(t)$ and $m(v)$, formulated as the following linear system:

$$\begin{bmatrix} \vec{r} & \vec{d} \end{bmatrix} \begin{bmatrix} v \\ t \end{bmatrix} = \mathbf{p} - C(u^*). \tag{5}$$

Consequently, our method decomposes Equation (2) with two unknowns u and v into two simpler ones, that is, Equations (4) and (5), based on geometric interpretation. These simpler equations can be easily solved sequentially, enabling efficient and robust computation of intersection points. Algorithm 1 shows how to compute the intersection lines between a triangle mesh and a ruled surface using the proposed method.

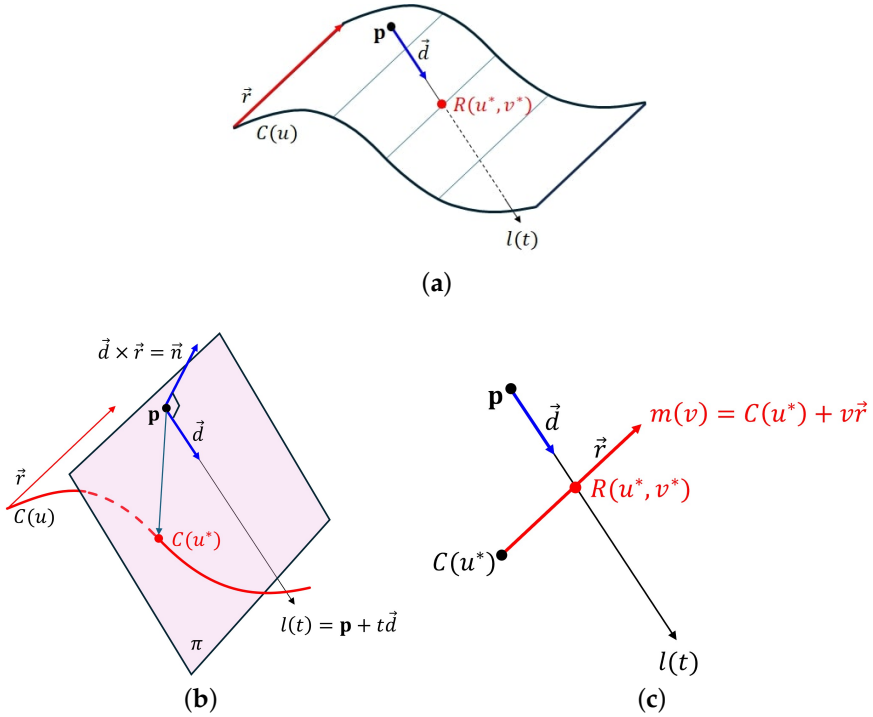


Figure 6. The LSI problem presented in (**a**) is broken down into two subproblems: (**b**) Plane–Curve Intersection (PCI) and (**c**) Line–Line Intersection (LLI).

Algorithm 1 Intersection of candidate triangles with a ruled surface

Require: $T = \{\Delta_i\}$: a set of candidate triangles, $R(u, v) = C(u) + v\vec{r}$: a ruled surface

for Δ_i **in** T **do**
  for e_j **in** Δ_i **do**
    $l_j(t) \leftarrow$ parametric line of e_j
    Create plane $\pi \leftarrow$ containing $l_j(t)$ and generator vector $\vec{r}$
    Compute intersection point $C(u^*)$ between π and $C(u)$
    $m(v) \leftarrow$ parametric line starting from $C(u^*)$
    Compute intersection point $R(u^*, v^*)$ between $l_j(t)$ and $m(v)$
  end for
end for

3.2 절단 (계속) · 경계 연결과 삼각형 분할

다음 단계는 계산한 교차점을 연결해 폴리라인을 만들고 메시를 여러 서브메시로 나누는 것이다. 이를 위해 삼각형과 직선생성곡면의 교차 가능성을 그림 7처럼 (1) 단일 교차와 (2) 두 점 교차로 분류한다. 퇴화된 절단 선분을 제거하기 위해 단일 교차(그림 7a)를 가지치기해야 한다. 퇴화 선분을 제거한 후 나머지 절단 선분을 연결하여 폴리라인을 만든다.

절단 선분은 곡면과 삼각형 에지의 교차점에서 만들어지므로, 폴리라인은 에지 위의 점들로 구성된다. 절단 선분이 포함된 모든 삼각형은 더 작은 두 개 또는 세 개의 삼각형으로 세분화·분할되어 서브메시를 형성한다(그림 8). 그림 8b처럼 새로운 서브메시는 열린 경계, 즉 구멍을 갖는다. 닫힌 입체가 되도록 다음 절에서 설명하는 단면으로 이 구멍들을 채워야 한다.

그림 7. 삼각형-곡면 교차: (a) 정점에서의 단일 교차, (b) 에지 위 두 점 교차, (c) 정점과 에지에서 각각 한 점씩 발생한 두 점 교차.

그림 8. 서브메시 생성: (a) 교차선으로 삼각형 세분화, (b) 열린 경계를 가진 두 서브메시.

The next step is to create polylines to subdivide the mesh into several submeshes by connecting the previously computed intersection points. To achieve this, we categorize the potential intersections of a triangle with a ruled surface into two types: (1) single intersections and (2) double intersections, as depicted in Figure 7. It is necessary to prune the single intersections (Figure 7a) to remove the degenerate cut segments. When the degenerate segments have been removed, the remaining cut segments are connected to form a polyline. This polyline consists of the points on the edges, as the cut segments are produced by the intersection points with the ruled surface on the triangle's edges. Every triangle that includes a cut segment is refined and split into two or three smaller ones, creating submeshes (see Figure 8). As shown in Figure 8b, these newly created submeshes have open boundaries (holes). To make sure the submeshes are solid, cross-sections need to be generated to fill these holes, which will be explained in the next subsection.

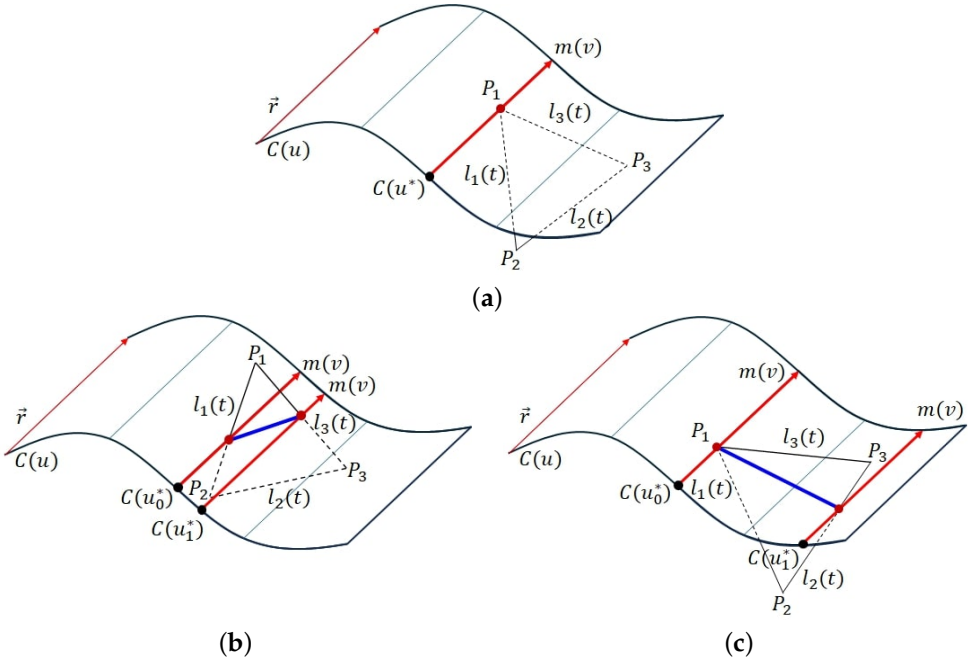


Figure 7. Triangle–Surface Intersections: (**a**) single intersection on a vertex, (**b**) two intersections on edges, and (**c**) two intersections on a vertex and an edge.

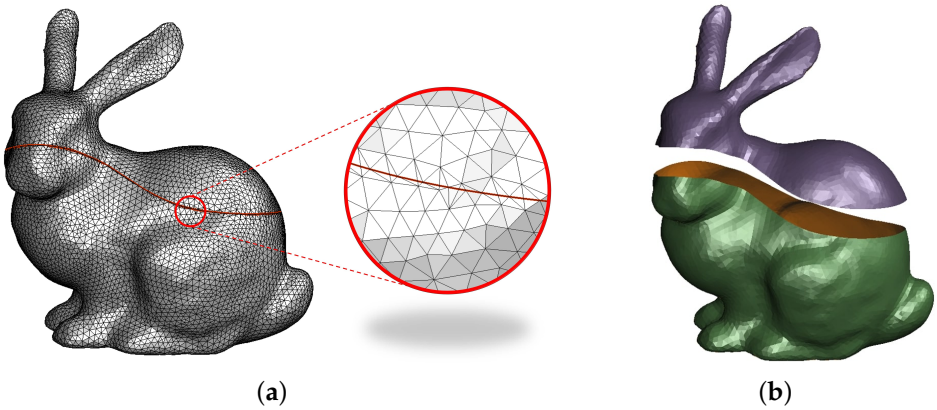


Figure 8. Submesh creation: (**a**) triangles are refined by intersection lines; (**b**) two submeshes with open boundaries.

3.3 단면 생성 및 병합

앞서 설명한 절단 방식은 열린 경계(구멍)를 가진 여러 서브메시를 만들 수 있다. 기존 구멍 메우기 기법[24-27]으로 구멍을 채울 수 있지만, 생성된 단면이 직선생성곡면의 단면과 완전히 일치하지 않을 수 있다. 이 문제를 해결하고 정확한 일치를 보장하려면 단면 패치를 직선생성곡면에서 직접 생성해야 한다.

제안 방법에서는 모든 교차점에 대응하는 매개변수 (u*,v*)가 곡면의 매개변수 영역 Ω = [0,1] × [0,1]에 존재한다. 그림 9b처럼 이 점들을 연결하면 Ω 안에 폴리라인을 쉽게 만들 수 있다. Ω에서 점들을 균일하게 샘플링하고, 제약 델로네 삼각분할[28]을 이용해 폴리라인으로 경계가 정해지는 부분 패치 φ(u,v)를 만든다. 그림 10a-d는 샘플 점 수가 다른 부분 패치 예시를 보여준다. 이에 대응하는 R(u,v)의 부분 패치는 X(u,v) = R · φ(u,v)로 정의되며, 그림 11처럼 서브메시의 열린 경계를 채우는 단면으로 사용한다.

그림 9. 절단 폴리라인: (a) 3D 공간, (b) 매개변수 영역 Ω.

그림 10. u,v 방향 샘플 점 수가 다른 부분 패치 φ(u,v): (a) 6 × 6, (b) 18 × 18, (c) 30 × 30, (d) 50 × 50.

Ω의 각 폴리라인에 대해 단면을 만든 후에는 서브메시가 닫힌 입체가 되도록 병합해야 한다. 단면과 서브메시를 그대로 병합하면 방향이 일치하지 않을 수 있다. 이를 해결하기 위해 두 경계의 방향을 비교한다. 양쪽 경계의 방향이 같으면 단면 방향이 서브메시와 불일치한다는 뜻이므로 뒤집어야 한다. 그렇지 않으면 그대로 병합하여 닫힌 서브메시를 만든다. 알고리즘 2는 전체 처리 과정을 나타낸다.

3.3. Cross-Section Creation and Merging

The ruled slicing method described above can result in multiple submeshes with open boundaries (holes). While conventional hole-filling techniques [24–27] can be utilized to fill these holes, the resulting cross-sections may not align perfectly with those of the ruled surface. To address this issue and ensure that the cross-sections precisely match the ruled surface, it is imperative to generate the cross-section patch directly from the ruled surface. In our method, all intersection points have corresponding parameters (u^*, v^*) in the parametric domain $\Omega = [0,1] \times [0,1]$ of a ruled surface. By connecting these points we can easily create a polyline in Ω, as illustrated in Figure 9b. We uniformly sample points in Ω and create a subpatch $\phi(u,v)$ bounded by the polyline using the constrained Delaunay triangulation [28]. Figure 10a–d shows examples of the generated subpatch $\phi(u,v)$ with varying numbers of sampling points. The corresponding subpatch $X(u,v)$ of $R(u,v)$ can be defined as $X(u,v) = R \cdot \phi(u,v)$ and is utilized as the cross-section to fill the open boundary of a submesh, as shown in Figure 11.

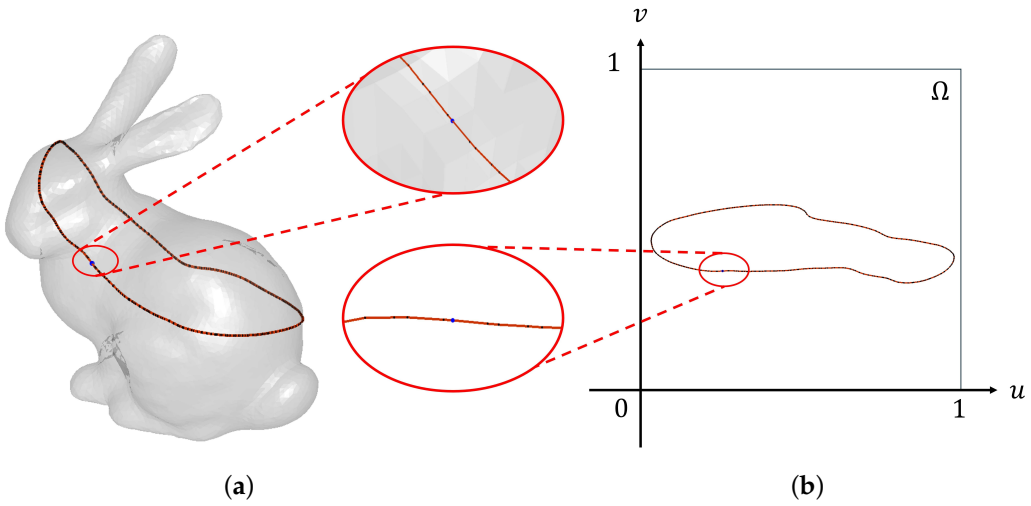


Figure 9. Cutting polyline in (**a**) 3D space and (**b**) the parametric domain Ω.

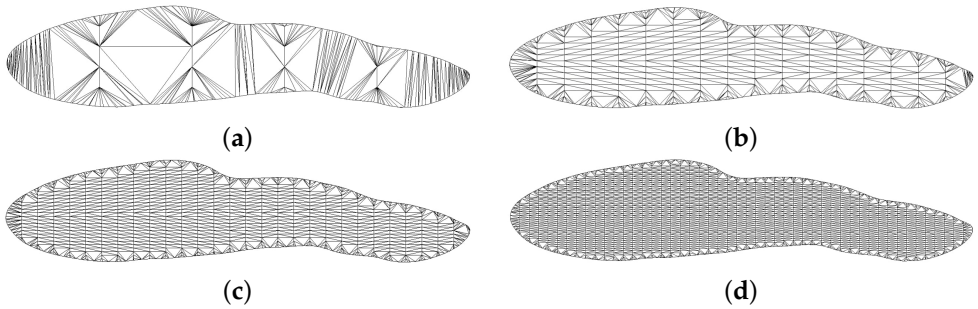


Figure 10. Subpatches $\phi(u,v)$ with different numbers of sampling points in u and v directions: (**a**) 6×6, (**b**) 18×18, (**c**) 30×30, (**d**) 50×50.

After a cross-section is created for each polyline in Ω, a merging step is necessary to ensure that the submeshes are solid. Merging the cross-section and submesh directly may lead to inconsistencies in orientation. To address this, we compare the boundary direction of the cross-section with that of the submesh. If both boundaries have the same direction, this indicates that the orientation of the cross-section is inconsistent with the submesh and must be flipped. If not, we simply merge them to make the submesh solid. Algorithm 2 illustrates the overall process of the proposed technique.

알고리즘 2 · 4. 실험 결과

그림 11. 직선생성곡면의 부분 패치로 Bunny 모델의 절단 경계 채우기.

알고리즘 2. 메시 절단 알고리즘

입력: 대상 메시 M, 자유곡선 C(u), 생성선 방향 벡터 r.

1. C(u)와 r로 직선생성곡면 R(u,v)를 생성한다.

2. R(u,v)의 BVH와 교차하는 M의 후보 삼각형 집합 T = {Δ_i}를 구한다.

3. T의 각 Δ_i에 대해 반복한다.

4. Δ_i와 R(u,v)의 교차 선분 s_i를 계산한다(알고리즘 1).

5. 반복을 종료한다.

6. 교차 선분을 연결하여 M 위의 폴리라인 집합 L = {l_i}를 만든다.

7. R(u,v)의 매개변수 영역에서 대응하는 폴리라인 집합을 만든다.

8. l_i를 이용해 M을 $M_1 \cup M_2 \cup \cdots \cup M_n$으로 분할한다.

9. 매개변수 영역의 폴리라인으로 R(u,v)를 잘라 단면 집합 X = {X_i}를 생성한다.

10. 각 서브메시 M_i에 대해 반복한다.

11. M_i의 각 경계 B_j에 대해 반복한다.

12. B_j에 대응하는 단면 X_k를 찾는다.

13. X_k의 법선 방향이 B_j와 불일치하면,

14. X_k의 법선 방향을 뒤집는다.

15. 조건문을 종료한다.

16. X_k를 B_j와 병합하여 경계를 채운다.

17. 경계 반복을 종료한다.

18. 서브메시 반복을 종료한다.

4. 실험 결과

제안 기법은 Intel i7-9700F CPU(3.00 GHz), 메모리 40.0 GB, NVIDIA GeForce RTX 2060 그래픽카드를 갖춘 PC에서 C++로 구현했다. 서로 다른 기하학적 특징을 갖는 다양한 3D 메시 모델에 직선생성곡면을 이용한 절단 실험을 수행했다. 표 1은 실험 모델의 기하학적 상세 정보를 보여준다.



Figure 11. The subpatch of a ruled surface fills the cutting boundary of the Bunny model.

Algorithm 2 Mesh slicing algorithm

Require: Target mesh M, freeform curve $C(u)$, and generator vector $\vec{r}$
$R(u,v) \leftarrow$ ruled surface generated by $C(u)$ and $\vec{r}$
$T = \{\Delta_i\} \leftarrow$ set of candidate triangles of M intersected with BVH of $R(u,v)$
for Δ_i **in** T **do**
    Compute intersection segment s_i of Δ_i with $R(u,v)$ (Algorithm 1)
end for
$L = \{l_i\} \leftarrow$ set of polylines on M generated by connecting intersection segments
$\hat{L} = \{\hat{l}_i\} \leftarrow$ set of polylines in the parametric domain of $R(u,v)$
Subdivide $M = M_1 \cup M_2 \cup \cdots \cup M_n$ using l_i
Create cross sections $X = \{X_i\}$ by cutting $R(u,v)$ with $\hat{l}_i$
for M_i **in** M **do**
    for B_j **in** boundaries of M_i **do**
        Find the cross-section X_k that matches with B_j
        if normal of X_k is inconsistent with B_j **then**
            Flip normal of X_k
        end if
        Fill B_j by merging X_k with B_j
    end for
end for

4. Experimental Results

The proposed technique was implemented using the C++ language on a PC equipped with an Intel i7-9700F CPU running at 3.00 GHz, 40.0 GB of memory and an NVIDIA GeForce RTX 2060 graphics card. Mesh slicing experiments using a ruled surface were conducted with various 3D mesh models exhibiting different geometric characteristics. Table 1 lists the geometric details of the models used in the experiment.

표 1 · 4.1 메시 절단 결과

표 1. 모델 통계(모델 이름과 원문 수치는 그대로 유지).

모델	정점 수	면 수	특징
Box	1,538	3,027	날카로운 에지
Knot	9,450	18,900	종수(genus) > 1
Bunny	8,327	16,650	복잡하고 매끄러운 표면
Camel	9,770	19,536	복잡하고 매끄러운 표면
Armadillo	43,243	86,462	복잡한 형상, 높은 잡음
Pumpkin Creature	58,657	117,310	복잡한 형상, 높은 잡음

4.1. 메시 절단 결과

제안 방법은 직선과 자유곡선이라는 두 종류의 스트로크를 제공한다. 직선의 경우 사용자가 마우스 클릭, 드래그, 놓기를 통해 시작점과 끝점을 정의한다. 이후 두 끝점을 통과하는 B-spline 곡선으로 직선생성곡면을 만든다. 생성된 곡면은 본질적으로 평면이지만 두 끝점으로 제한되므로 그림 12처럼 원하는 영역을 정확하게 절단할 수 있다.

그림 13은 직선 절단의 다양한 결과를 보여준다. 파란 점선은 사용자 스트로크, 빨간 실선은 메시 위의 절단 폴리라인이다. 파란 선은 메시와 여러 번 교차하여 여러 서브메시를 만들 수 있다. 그림 14는 자유곡선이 대상 메시를 한 번 가로질러 각각 두 서브메시가 생기는 결과를 보여준다. 그림 15처럼 자유곡선이 대상 메시와 여러 번 교차하여 여러 서브메시를 생성하는 경우도 지원한다. 실시간 시연은 첨부 영상 또는 Video S1을 참조한다: https://youtu.be/UqQaKZTj_Zo (원문의 접속일: 2024년 4월 23일).

그림 12. 동일한 스트로크 (a)에 대한 절단 결과 비교: (b) 제안 방법, (c) 기존 평면.

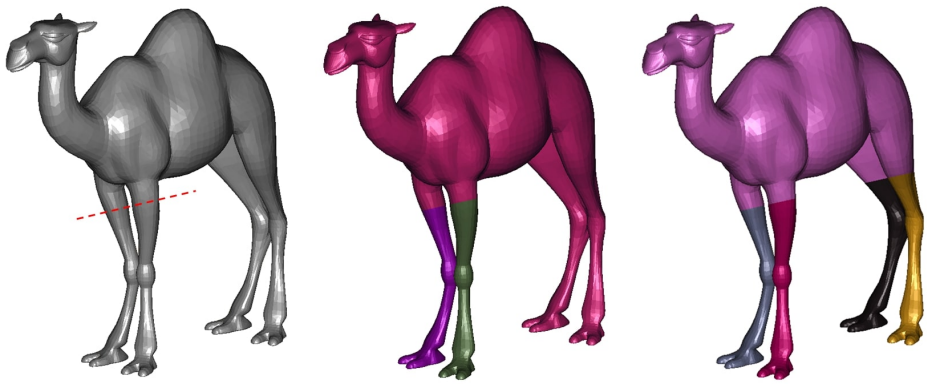
사용자 스트로크는 자유곡선으로 근사되고, 이 곡선에서 만든 곡면으로 대상 메시가 절단된다. 그러나 일부 사용자는 생성된 자유곡선을 편집하여 결과를 더 정교하게 제어하고 싶을 수 있다. 이를 위해 절단 곡선을 수정하는 기능을 제공한다. 그림 16은 수동 편집 예시이다. 사용자가 화면에 경로를 그리면 초기 근사 곡선을 생성하며, 초록색 원으로 표시된 제어점을 조정하면 사용자 의도에 맞게 곡선이 수정된다.

Table 1. Statistics of the models.

Models	Box	Knot	Bunny	Camel	Armadillo	Pumpkin Creature
# of vertices	1538	9450	8327	9770	43,243	58,657
# of faces	3027	18,900	16,650	19,536	86,462	117,310
Feature	Sharp edges	Genus > 1	Complex Smooth surface	Complex Smooth surface	Complex High noise	Complex High noise

4.1. Mesh Slicing Results

Our method offers users two types of strokes: a straight line and a freeform curve. For a straight line, the user defines the start and end points of the cutting line by mouse clicking, dragging, and releasing. Subsequently, a ruled surface is generated from the B-spline curve that passes through two end points. It is important to note that although the generated ruled surface is essentially a plane, it is confined by the two end points; therefore, our method enables precise slicing of a desired region as shown in Figure 12. Various results of mesh slicing using straight lines are illustrated in Figure 13, where the blue dashed lines denote the user's strokes and the red solid lines correspond to the cutting polylines on the meshes. Note that the blue line may intersect with the mesh multiple times, resulting in multiple submeshes. Figure 14 shows the results of mesh slicing using freeform curves, where the curves intersect with the target meshes once, resulting in two submeshes, respectively. Our method allows a freeform curve to intersect with the target mesh multiple times, generating multiple submeshes, as shown in Figure 15 (refer to the attached video for a real-time demonstration (https://youtu.be/UqQaKZTj_Zo (accessed on 23 April 2024) or see the Video S1).



(**a**) (**b**) (**c**)

Figure 12. Comparison of slicing results between (**b**) our method and (**c**) conventional plane for the same user's stroke in (**a**).

The user's stroke is approximated by a freeform curve, then the target mesh is sliced by the ruled surface generated from the freeform curve. However, certain users may prefer more sophisticated control over the slicing result by editing the generated freeform curve. To accommodate this, our method provides the flexibility for users to modify the slicing curve. Figure 16 illustrates an example of manual editing of the generated freeform curve. As the user draws the slicing trajectory on the screen, the initial approximation curve is generated. By adjusting the control points (the green circle), the curve is modified as the user intends.

그림 13 · 그림 14

그림 13. 직선 스트로크를 사용한 다양한 메시 절단 결과.

그림 14. 자유곡선으로 메시를 두 서브메시로 나눈 다양한 결과.

이 페이지는 원문에서도 그림 위주로 구성되어 있다. 대응하는 그림 전체를 다음 원문 페이지에 보존했다.

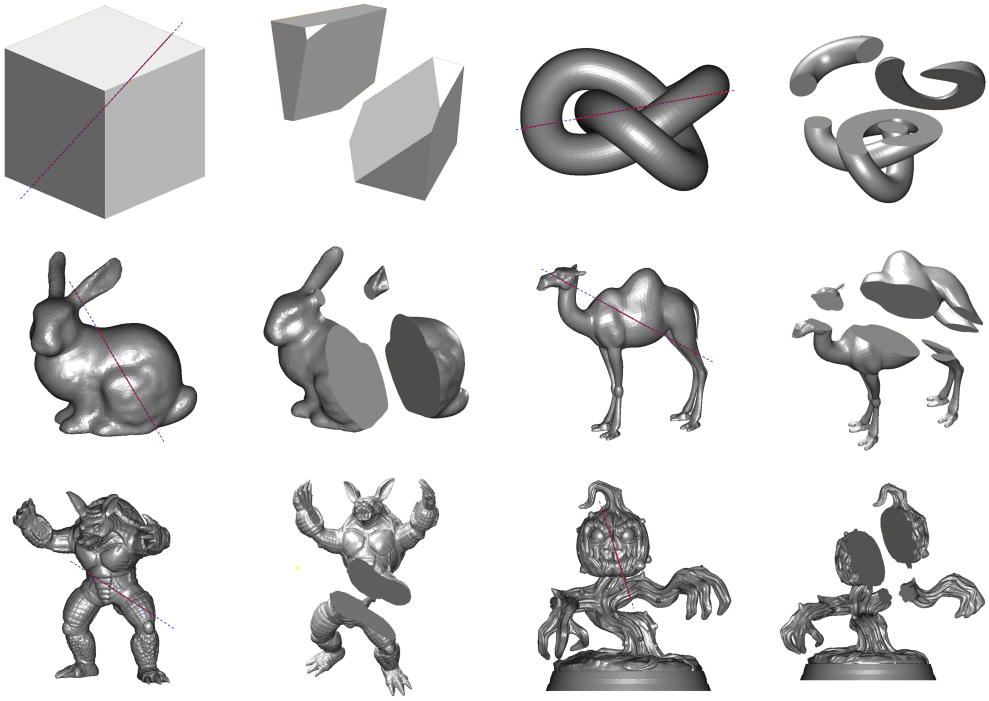


Figure 13. Various mesh slicing results using straight line strokes.

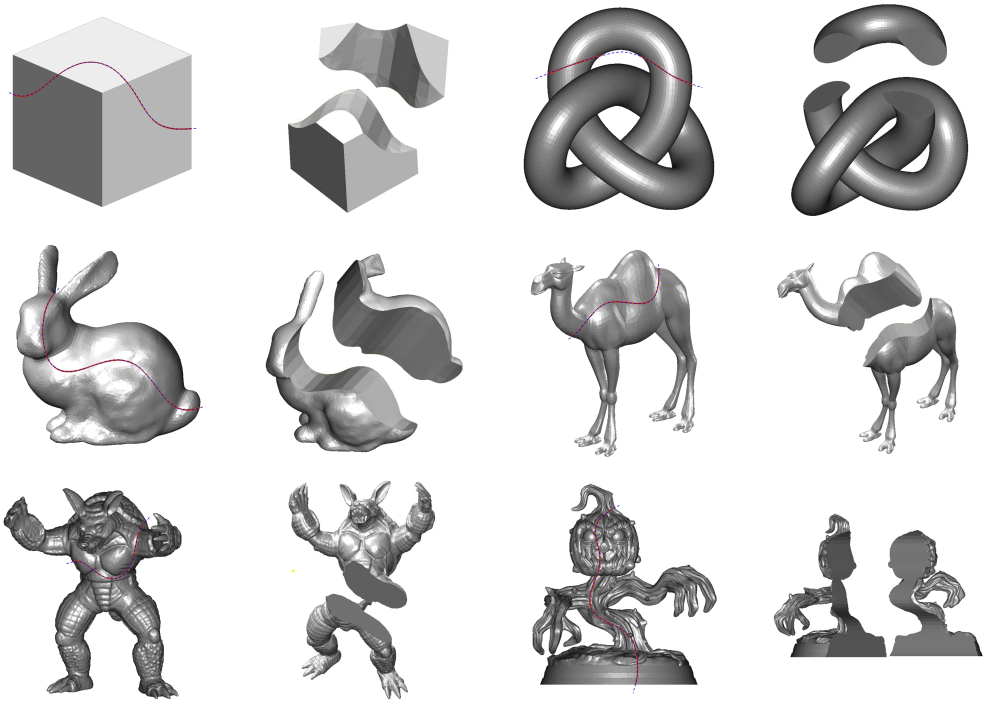


Figure 14. Various results of slicing meshes into two submeshes using freeform curves.

그림 15 · 그림 16 · 4.2 성능 분석

그림 15. 자유곡선으로 메시를 여러 서브메시로 나눈 다양한 결과.

그림 16. 절단 곡선 편집: (a) 사용자의 자유 스트로크, (b) 초기 근사 B-spline 곡선, (c) 절단 곡선의 제어점 편집.

4.2. 성능 분석

표 2는 그림 13-15의 각 절단 과정 단계에 대응하여 여러 메시를 서브메시로 나누는 소요 시간을 보여준다. 각 단계의 시간을 합산하여 전체 소요 시간을 계산한다. 모델들의 특징이 다양함에도 대부분의 시간은 교차 계산, 서브메시 생성, 병합 단계에서 발생했다. 정점과 면 수가 훨씬 많은 Armadillo와 Pumpkin Creature 같은 모델에서도 대부분의 실험은 약 1-2초 안에 만족스러운 결과를 얻었다. 이 관찰은 소요 시간이 모델의 복잡도보다 표에 나열된 후보 면의 수에 주로 영향을 받는다는 점을 시사한다.

계산 효율을 보이기 위해 제안 방법(PCI와 LLI 분해)과 식 (2)의 일반적인 LSI 계산 시간을 비교했다. 다양한 모델에서 메시 절단을 100번 수행하고 각 방법의 평균 시간을 계산했다. 그림 17에서 파란 선은 제안 방법, 주황 선은 일반 LSI이다. 제안 방법의 소요 시간은 일반 LSI보다 상당히 짧다. 후보 면 수가 증가할수록 두 방법의 차이는 더 커진다.

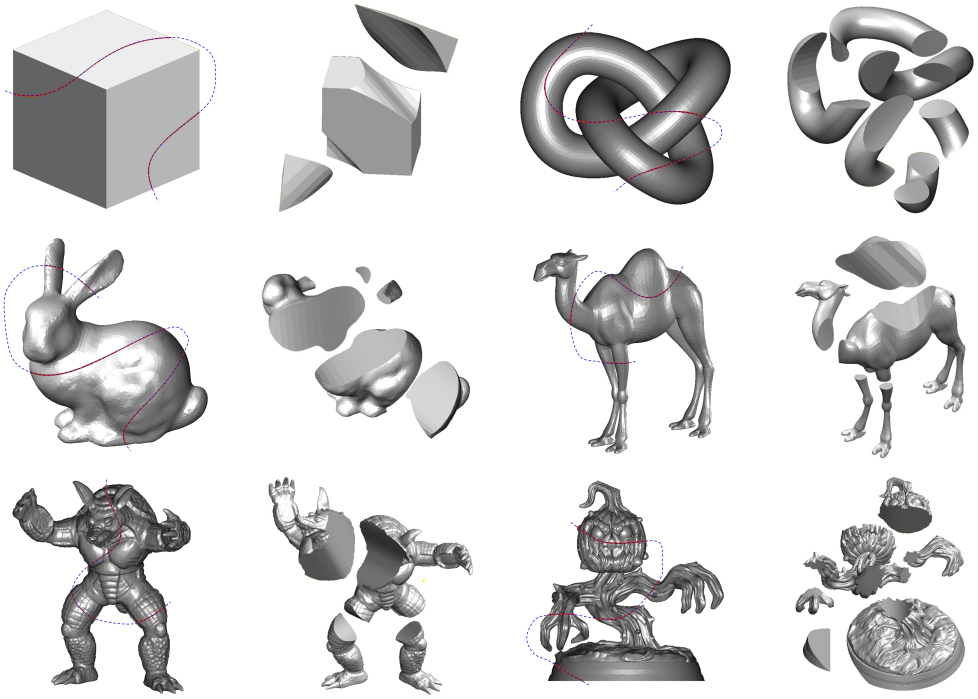


Figure 15. Various results of slicing meshes into multiple submeshes using freeform curves.

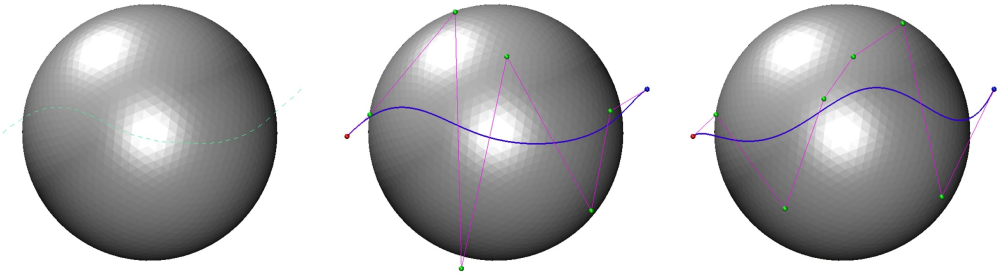


Figure 16. Editing slicing curve: (**a**) user's free stroke, (**b**) initial approximation B-spline curve, and (**c**) editing control points of a slicing curve.

4.2. Performance Analysis

Table 2 lists the elapsed times for slicing various meshes into submeshes, corresponding to each step of the slicing process depicted in Figures 13–15. The elapsed time for each step is then summed to determine the total elapsed time. Regardless of the diverse features among the models, the majority of the elapsed time is measured on the intersection computation, submesh creation, and merging steps. Even in the case of models such as the Armadillo and Pumpkin Creature, which have significantly larger numbers of vertices and faces, most experiments yielded satisfactory results within approximately 1 to 2 s. This observation suggests that the elapsed time is primarily influenced by the number of candidate faces listed in the third row, rather than the complexity of the model.

To demonstrate its computational efficiency, we compared the elapsed times of our method (decomposition into PCI and LLI) with a general LSI computation in Equation (2). We performed mesh slicing 100 times on various models and computed the average elapsed time for each method. Figure 17 shows the comparative results of the two methods. The blue line corresponds to our method and the orange line to general LSI computation. The elapsed time of our method is significantly less than that of the general LSI method. Moreover, the difference between the two methods becomes larger as the number of candidate faces

표 2 · 그림 17 · 4.3 기존 방법과 비교

이는 제안 방법이 일반 LSI보다 훨씬 빠르고 효율적임을 확인한다.

표 2. 메시 절단 알고리즘의 소요 시간(ms). 후보 면 수는 개수이다.

단계 / 모델	Box	Knot	Bunny	Camel	Armadillo	Pumpkin Creature
직선생성곡면 생성	0	0	0	0	0	0
후보 면 수	387	609	457	578	1,170	3,050
교차 계산	73	80	88	159	230	499
서브메시 생성	21	84	32	133	530	1,183
단면 생성	2	2	2	4	4	12
법선 보정	1	3	1	3	7	14
단면 병합	13	41	19	87	298	506
전체	110	210	142	386	1,069	2,214

그림 17. LSI와 제안 방법의 효율 비교. 가로축은 후보 면 수, 세로축은 소요 시간(ms).

4.3. 기존 방법과 비교

단면 생성의 유효성을 보이기 위해 경계를 고정한 상태에서 최소 곡면으로 구멍을 채우는 기존 표면 페어링 기법[24]과 비교했다. 그림 18은 동일한 사용자 스트로크에 대해 최소 곡면과 제안 방법이 생성한 단면을 보여준다. 그림 18a처럼 표면 페어링 기반 구멍 메우기는 원하는 절단면에 적합하지 않다. 반면 제안 방법은 직선생성곡면을 직접 잘라 단면을 만들므로 그림 18b처럼 절단 단면과 정확히 일치한다. 이는 임의의 경계 배치를 가진 정확한 단면을 생성할 수 있다는 장점을 강조한다.

추가로 자유곡선을 선분 집합으로 이산화하는 방식과 곡선-삼각형 메시 교차점을 직접 계산하는 방식의 결과를 비교했다. 그림 19b,c에서는 선분 끝점에서 삼각형이 세분화되어 과도한 복잡도와 불필요하게 세분화된 삼각형이 나타난다. 반면 제안 방법은 교차점에서만 삼각형을 세분화하므로 불필요한 세분화를 피하고, 그림 19a처럼 간결하고 최적화된 삼각형을 생성한다.

절단 결과 품질을 정량적으로 비교하기 위해 표 1의 각 모델을 동일한 자유곡선으로 절단했다. 표 3은 제안 방법과 서로 다른 샘플 수의 이산화 방법이 생성한 단면 경계의 교차점(정점) 및 퇴화 삼각형 수를 보여준다. 이산화 방법은 샘플 점 수가 증가할수록 더 많은 교차점과 퇴화 삼각형을 생성한다.

increases. This confirms that our method is much faster and more efficient than the general LSI method.

Table 2. Elapsed time (in ms) of mesh slicing algorithm.

Models	Box	Knot	Bunny	Camel	Armadillo	Pumpkin Creature
Ruled surface creation	0	0	0	0	0	0
# of Candidate Faces	387	609	457	578	1170	3050
Intersection computation	73	80	88	159	230	499
Submesh creation	21	84	32	133	530	1183
Cross-section creation	2	2	2	4	4	12
Normal correction	1	3	1	3	7	14
Merging cross-section	13	41	19	87	298	506
Total	110	210	142	386	1069	2214

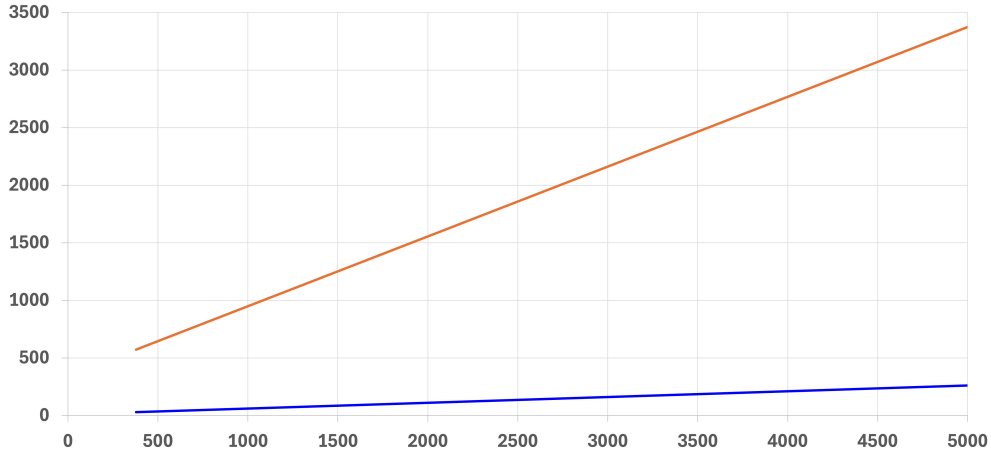


Figure 17. Efficiency comparison between the LSI and our methods. The horizontal axis represents the number of candidate faces and the vertical axis represents the elapsed time (ms).

4.3. Comparison to Existing Methods

To demonstrate the effectiveness of our method for generating cross-sections, we compared our method with the state-of-the-art surface fairing technique from [24], which fills holes with a minimal surface while keeping its boundary fixed. For the user's identical strokes, Figure 18 shows the cross-sections generated by the minimal surface and our method, respectively. As shown in Figure 18a, the hole-filling method based on the surface fairing is not suitable for the desired cross-section, whereas our method produces the cross-section by directly cutting the ruled surface. Therefore, as shown in Figure 18b, it exactly matches the slicing cross-section. This effectiveness highlights the advantage of our method, which allows for the generation of accurate cross-sections with arbitrary boundary configurations.

We additionally compared the mesh slicing results by discretizing the freeform curve into a set of line segments and directly computing the intersection points between the curve and the triangular mesh. Figure 19 demonstrates the effectiveness of our method. Figure 19b,c shows excessive complexity and unnecessary refined triangles due to the refinement of the triangles at the endpoints of the line segments. On the other hand, our method refines the triangles only at the intersection points while avoiding unnecessary refinements, and produces concise and optimal triangles, as shown in Figure 19a.

To quantitatively compare the quality of the slicing results generated using our method and the discretization method, we sliced each model in Table 1 using an identical freeform curve. Table 3 lists the number of intersection points (vertices) and degenerate triangles at the boundaries of the cross-sections generated by our method and the discretization method with different numbers of sample points. The discretization method generates more intersection points and degenerate triangles as the number of sample points increases,

4.3 비교 (계속) · 표 3 · 4.4 응용

따라서 그림 19b,c처럼 단면의 기하학적 복잡도가 높아진다. 반대로 제안 방법은 교차점과 퇴화 삼각형을 더 적게 생성하여 더욱 효과적이고 강건한 절단 결과를 만든다.

그림 18. 단면 생성 결과 비교: (a) 표면 페어링 방법, (b) 제안 방법.

그림 19. 메시 절단 결과: (a) 교차점 계산 방식, (b) 30개 선분 사용, (c) 100개 선분 사용.

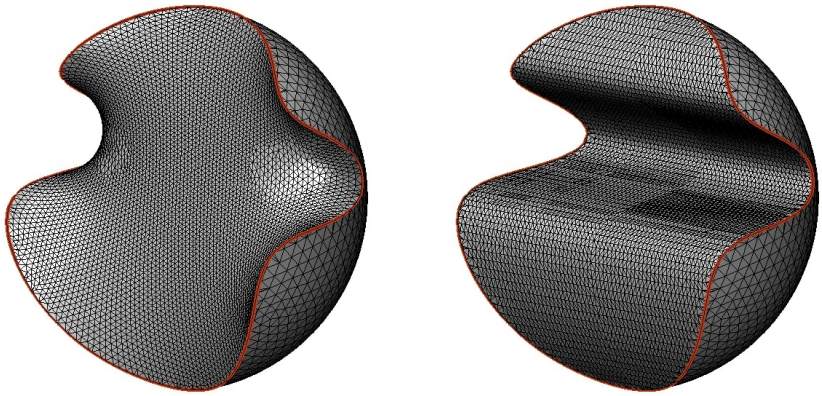
표 3. 제안 방법과 이산화 방법의 교차점 및 퇴화 삼각형 수 비교. 면적이 $\varepsilon = 10^{-3}$보다 작은 삼각형을 퇴화 삼각형으로 분류한다.

모델	교차점: 제안	교차점: 30	교차점: 100	퇴화 면: 제안	퇴화 면: 30	퇴화 면: 100
Box	207	343	661	0	8	12
Knot	447	583	911	0	12	18
Bunny	365	470	715	4	21	38
Camel	373	494	795	0	2	16
Armadillo	859	973	1,238	5	13	10
Pumpkin Creature	1,182	1,320	1,683	33	54	60

4.4. 응용

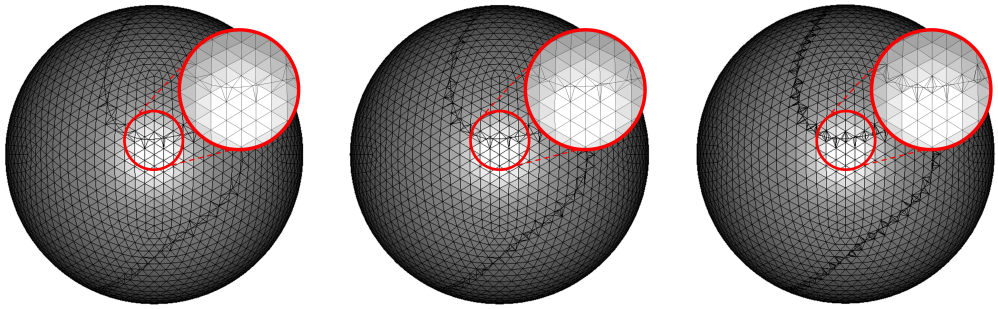
제안 방법은 사용자 맞춤형 3D 퍼즐 생성 같은 다양한 응용에 쉽게 적용할 수 있다. 예시로 그림 20처럼 Bunny 모델을 여덟 조각으로 나누었다. 그림 20a는 절단 결과이고, 그림 20b는 녹인 필라멘트를 층별로 쌓아 만드는 FDM 방식으로 출력한 3D 퍼즐 조각이다.

resulting in higher geometric complexity of the cross-sections, as shown in Figure 19b,c. Conversely, our method produces fewer intersection points and degenerate triangles, resulting in more effective and robust slicing results.



(**a**) (**b**)

Figure 18. Comparison of cross-section generation results using (**a**) surface fairing method and (**b**) our method.



(**a**) (**b**) (**c**)

Figure 19. Mesh slicing results: (**a**) computing intersection points, (**b**) using a set of 30 line segments, and (**c**) using a set of 100 line segments.

Table 3. Comparison of the number of intersection points and degenerate triangles between our method and the discretization method (triangles with an area less than $\epsilon = (10^{-3})$ are classified as degenerate triangles).

Models	# of Intersection Points			# of Degenerate Faces		
	Our	Discretization (30 Samples)	Discretization (100 Samples)	Our	Discretization (30 Samples)	Discretization (100 Samples)
Box	207	343	661	0	8	12
Knot	447	583	911	0	12	18
Bunny	365	470	715	4	21	38
Camel	373	494	795	0	2	16
Armadillo	859	973	1238	5	13	10
Pumpkin Creature	1182	1320	1683	33	54	60

4.4. Application

Our method can be easily applied in various applications, such as custom 3D puzzle generation. As an example, we sliced the Bunny model into eight pieces, as shown in Figure 20. Figure 20a shows the results of slicing the Bunny model, while Figure 20b shows

4.4 응용 (계속) · 5. 결론 및 논의

출력 과정에서 약간의 오차가 생길 수 있지만, 만들어진 조각은 만족스러운 결과를 제공한다. 조각을 조립하면(그림 20c) 원래 Bunny 모델이 완전히 재구성된다. 제조 분야로 확장하면 더 정교하고 복잡한 3D 퍼즐을 만드는 데 도움이 될 수 있다.

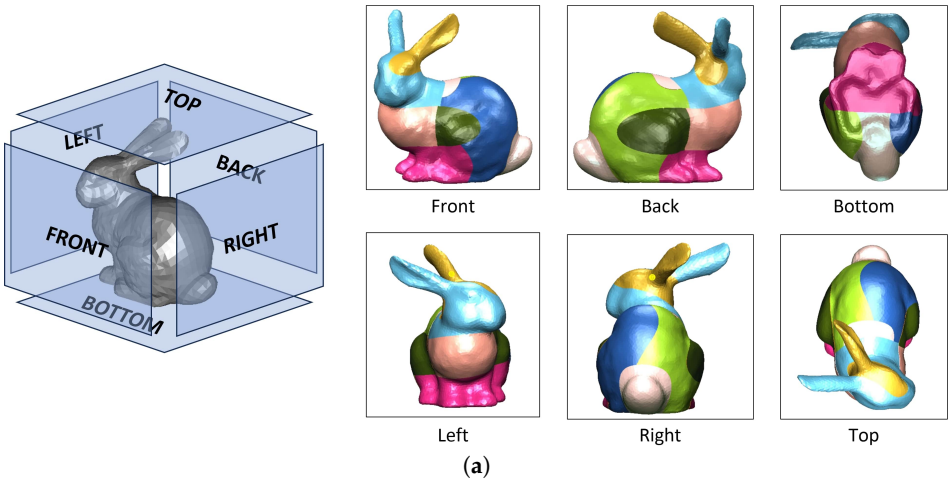
그림 20. 3D 퍼즐 응용: (a) 제안 방법에 의한 절단 결과, (b) 출력한 3D 퍼즐 조각, (c) 재구성한 3D 모델.

5. 결론 및 논의

본 논문은 자유곡선으로 삼각형 메시를 절단하는 효율적인 방법을 소개했다. 사용자 스트로크로부터 생성한 직선생성 곡면을 활용하며, 서로 다른 특징을 갖는 다양한 메시에서 성공적으로 실험했다. 사용자가 화면에 스트로크를 그리면 B-spline 곡선으로 근사하고, 시선 방향으로 압출하여 곡면을 생성한다.

효율적인 절단을 위해 곡면의 BVH로 교차 후보 면을 찾는다. 후보 면이 확인되면 곡면과 후보 면의 정확한 교차점을 계산한다. LSI를 PCI와 LLI로 분해하여 교차 매개변수 u,v를 효율적으로 계산하고 정확도와 강건성을 높인다. 교차점을 연결하여 폴리라인을 만들고 메시를 여러 서브메시로 절단한다. 서브메시가 닫힌 입체가 되도록 폴리라인을 따라 곡면을 트리밍해 단면을 생성하고 해당 서브메시와 병합한다. 기존 방법과의 비교 및 다양한 절단 사례로 제안 방법의 유효성을 보였다.

the 3D printed puzzle pieces using a fused deposition modeling (FDM) method, where the melted filament is stacked layer-by-layer to form the 3D model. While this process can introduce slight errors, the resulting pieces still provide satisfying results. When assembled (Figure 20c), these pieces perfectly reconstruct the original Bunny model. Expanding this application to the manufacturing industry could facilitate the creation of even more intricate and highly sophisticated 3D puzzles.



(**a**)



(**b**) (**c**)

Figure 20. Application to 3D puzzle scenario: (**a**) mesh slicing results using our method, (**b**) 3D printed puzzle pieces, and (**c**) reconstructed 3D model.

5. Conclusions and Discussion

This paper introduces an efficient method for slicing a triangle mesh using a freeform curve. The method utilizes a ruled surface generated based on the user's stroke, and has been successfully tested on various triangle meshes with different characteristics. The user interaction involves drawing a stroke on the screen, which is then approximated by a B-spline curve. Subsequently, the curve is extruded in the viewing direction, creating a ruled surface. To achieve efficient slicing, we leverage the BVH of the ruled surface to identify candidate faces for potential intersections. When the candidate faces have been identified, we compute the exact intersection points between the ruled surface and these faces. This process is efficiently performed by decomposing the LSI problem into PCI and LLI problems. This decomposition allows for efficient computation of the intersection parameters u and v, improving both accuracy and robustness. Intersection points are then connected to form polylines, effectively cutting the mesh into multiple submeshes. To ensure the solidity of the submeshes, cross-sections are generated by trimming the ruled surface along the polylines, then merged with the corresponding submeshes. We have demonstrated the effectiveness of the proposed method by comparing it with existing methods and showing various examples of mesh slicing.

5. 결론 (계속) · 보충자료 · 저자·지원 정보 · 참고문헌 1-15

제안 접근은 여러 장점이 있다. 첫째, 자유곡선을 편집하여 직선과 자유롭게 휘어진 경로를 포함하는 임의의 경로로 삼각형 메시를 절단할 수 있다. 둘째, 한 스트로크당 두 조각으로 제한하는 기존 방법과 달리 메시 위상과 관계없이 여러 서브메시로 나눌 수 있다. LSI를 PCI와 LLI로 분해하면 교차 매개변수를 따로 계산할 수 있어 효율성과 강건성이 높아진다. 마지막으로 기존 방법이 구멍 메우기에 의존하는 데 비해 단면을 직선생성곡면에서 쉽게 생성한다.

그러나 현재 구현은 볼륨 메시를 처리하지 못한다. 사면체 같은 볼륨 요소는 삼각형보다 훨씬 많은 교차점을 포함하여 계산 비용이 기하급수적으로 증가하기 때문에 큰 과제이다. 향후에는 GPU 기반 병렬 프로그래밍을 도입하여 효율을 향상시키고 볼륨 메시의 실시간 절단을 가능하게 하고자 한다.

보충자료: https://www.mdpi.com/article/10.3390/math12101432/s1 에서 Video S1: Experimental_Results를 받을 수 있다.

저자 기여: S.-Y.L.과 S.-H.K.이 실험을 구상하고 설계했다. S.-Y.L.이 제안 기법을 구현하고 실험을 수행했다. S.-H.Y.이 논문을 작성했다. 모든 저자는 출판 원고를 읽고 동의했다.

연구비 지원: 대한민국 정부(과학기술정보통신부)가 지원하는 IITP의 인공지능융합혁신인재양성 사업 (IITP-2024-RS-2023-00254592)의 지원을 받았다.

데이터 가용성: 데이터는 논문 안에 포함되어 있다.

이해충돌: 저자들은 이해충돌이 없음을 선언한다.

참고문헌 1-15: 제목 한국어 번역

정확한 저자·연도·출판 정보는 다음 원문 페이지의 참고문헌을 유지했다.

1. 표면 재메시: 방법과 연구 방향에 대한 체계적 문헌 검토.

2. 유한요소를 위한 일반적인 메시 평활화 방법.

3. 구조를 고려한 메시 단순화.

4. 메시 변형 접근법: 서베이.

5. 3D 모델의 부분 메시 절단 방법을 이용한 원자로 해체 시뮬레이션.

6. 손으로 그린 스트로크로 메시 모델을 절단하는 구현하기 쉬운 방법.

7. Teddy: 3D 자유형 설계를 위한 스케치 인터페이스.

8. 볼륨 일러스트레이션: 내부 텍스처를 갖는 3D 모델 설계.

9. 다변량 유리 스플라인 함수를 이용한 기하 제약조건 해결기.

10. 3D 프린터의 출력 볼륨보다 큰 물체를 출력하기 위한 상호작용형 3D 메시 편집 시스템.

11. 점진적 메시 절단 알고리즘을 이용한 3차원 의복 패턴 설계.

12. 3D 형상의 내부 구조 모델링을 위한 스케치 인터페이스.

13. 다양한 세분화 수준의 위치 기반 동역학을 이용한 Unity 3D 절단 시뮬레이션.

14. 고성능 가상현실에서 연성 물체의 점진적 찢김과 절단.

15. 혼합현실에서 3D 모델을 위한 상호작용형 실시간 절단 기법.

Our approach offers several advantages. First, our method enables users to slice triangle meshes along arbitrary trajectories, encompassing both straight and freely curved paths by editing the freeform curves. Second, unlike existing methods that limit mesh slicing to two pieces per stroke, our approach allows for slicing the mesh into multiple submeshes regardless of the mesh topology. The decomposition of LSI into PCI and LLI further improves the efficiency and robustness of the method by enabling separate computations of intersection parameters. Finally, our cross-sections can be easily generated from the ruled surface, while existing methods rely on a hole-filling method.

Despite the numerous advantages mentioned above, the current implementation is not able to handle volumetric meshes. This poses a significant challenge, as volume elements such as tetrahedra involve a considerably greater number of intersection points compared to triangles, leading to an exponential increase in computational cost. In future work, we aim to enhance the efficiency of our technique by incorporating GPU-based parallel programming, enabling real-time slicing of volumetric meshes.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/math12101432/s1, Video S1: Experimental_Results.

Author Contributions: S.-Y.L. and S.-H.K. conceived and designed the experiments; S.-Y.L. implemented the proposed technique and performed the experiments; S.-H.Y. wrote the paper. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the Artificial Intelligence Convergence Innovation Human Resources Development (IITP-2024-RS-2023-00254592) grant funded by the Korea government (MSIT).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Khan, D.; Plopski, A.; Fujimoto, Y.; Kanbara, M.; Jabeen, G.; Zhang, Y.J.; Zhang, X.; Kato, H. Surface remeshing: A systematic literature review of methods and research directions. *IEEE Trans. Vis. Comput. Graph.* **2020**, *28*, 1680–1713. [CrossRef]
2. Durand, R.; Pantoja-Rosero, B.; Oliveira, V. A general mesh smoothing method for finite elements. *Finite Elem. Anal. Des.* **2019**, *158*, 17–30. [CrossRef]
3. Salinas, D.; Lafarge, F.; Alliez, P. Structure-aware mesh decimation. In *Computer Graphics Forum*; Wiley Online Library: Hoboken, NJ, USA, 2015; Volume 34, pp. 211–227.
4. Selim, M.; Koomullil, R. Mesh deformation approaches—A survey. *J. Phys. Math.* **2016**, *7*, 1–9.
5. Lee, W.B.; Hao, W.Y.; Kyung, B.P.; Ryu, S.H. Dismantling Simulation of Nuclear Reactor Using Partial Mesh Cutting Method for 3D Model. *J. Digit. Converg.* **2015**, *13*, 303–310.
6. Mitani, J. A simple-to-implementation method for cutting a mesh model by a hand-drawn stroke. In Proceedings of the EUROGRAPHICS Workshop on Sketch-Based Interfaces and Modeling, Trinity College Dublin, Dublin, Ireland, 28–29 August 2005.
7. Igarashi, T.; Matsuoka, S.; Tanaka, H. Teddy: A sketching interface for 3D freeform design. In *ACM SIGGRAPH 2006 Courses*; Association for Computing Machinery: New York, NY, USA, 2006; p. 11-es.
8. Owada, S.; Nielsen, F.; Okabe, M.; Igarashi, T. Volumetric illustration: Designing 3D models with internal textures. In *ACM SIGGRAPH 2004 Papers*; Association for Computing Machinery: New York, NY, USA, 2004; pp. 322–328.
9. Elber, G.; Kim, M.S. Geometric constraint solver using multivariate rational spline functions. In Proceedings of the Sixth ACM Symposium on Solid Modeling and Applications, New York, NY, USA, 4–8 June 2001; pp. 1–10. [CrossRef]
10. Chu, C.W.; Kim, K.K.; Park, C.J.; Choi, J.S. An Interactive 3D Mesh Editing System for Printing Object Larger Than the Printing Volume of 3D Printer. *J. Korea Multimed. Soc.* **2016**, *19*, 1619–1625. [CrossRef]
11. Kang, Y.; Kim, S. Three-dimensional garment pattern design using progressive mesh cutting algorithm. *Int. J. Cloth. Sci. Technol.* **2019**, *31*, 339–349. [CrossRef]
12. Owada, S.; Nielsen, F.; Nakazawa, K.; Igarashi, T. A sketching interface for modeling the internal structures of 3d shapes. In *ACM SIGGRAPH 2006 Courses*; Association for Computing Machinery: New York, NY, USA, 2006; p. 12-es.
13. Khan, L.; Choi, Y.J.; Hong, M. Cutting simulation in unity 3D using position based dynamics with various refinement levels. *Electronics* **2022**, *11*, 2139. [CrossRef]
14. Kamarianakis, M.; Protopsaltis, A.; Angelis, D.; Tamiolakis, M.; Papagiannakis, G. Progressive tearing and cutting of soft-bodies in high-performance virtual reality. *arXiv* **2022**, arXiv:2209.08531.
15. Caligiana, P.; Liverani, A.; Ceruti, A.; Santi, G.M.; Donnici, G.; Osti, F. An interactive real-time cutting technique for 3D models in mixed reality. *Technologies* **2020**, *8*, 23. [CrossRef]

참고문헌 16-28 · 출판사 면책문

참고문헌 16-28: 제목 한국어 번역

정확한 저자·연도·출판 정보는 다음 원문 페이지에 보존했다.

16. 평면-곡면 교차 알고리즘과 조선 분야 응용.

17. 매개변수 Bézier 삼각형 곡면과 평면의 교차를 위한 추적 방법.

18. 직선생성곡면과 평면 사이의 교차 곡선을 계산하는 효율적인 방법.

19. NURBS 곡면의 기하 연산 알고리즘.

20. 트리밍된 NURBS 곡면의 실용적인 광선 추적.

21. 유리 Bézier 곡선 및 곡면의 음함수 행렬 표현.

22. 행렬 표현을 이용한 직선과 트리밍된 NURBS 곡면의 교차 알고리즘.

23. 기준 곡면을 따라 추적하는 점 역변환 및 광선-곡면 교차 계산.

24. 적층 보수에서 삼각형 메시를 위한 빠른 구멍 메우기 방법.

25. 형상 차이에 기반한 구멍 패치의 매개변수 블렌딩.

26. 역설계에서 조각별 구멍 메우기 알고리즘.

27. 삼각형 메시를 위한 강건한 구멍 메우기 알고리즘.

28. 2D 제약 델로네 삼각분할 알고리즘의 최근 발전.

출판사 면책문

모든 출판물에 포함된 진술, 의견, 데이터는 개별 저자와 기여자의 것이며 MDPI 및 편집자의 것이 아니다. MDPI와 편집자는 본문에 언급된 아이디어, 방법, 지침 또는 제품으로 인해 발생하는 인명·재산 피해에 대해 책임을 지지 않는다.

16. Lu, C.; Lin, Y.; Ji, Z.; Dalian Univ. of Technology. An algorithm for plane-surface intersection and its application to shipbuilding. *Ship Technol. Res.* **2005**, *52*, 103–106. [CrossRef]
17. Sharma, R.; Sha, O. A tracing method for parametric Bezier triangular surface/plane intersection. *Int. J. Comput. Appl. Technol.* **2007**, *28*, 240–253. [CrossRef]
18. Li, X.; Jiang, S.; Huang, B.; Zeng, Q. An Efficient Method for Calculating the Intersection Curve between a Ruled Surface and a Plane. *Int. J. Signal Process. Image Process. Pattern Recognit.* **2013**, *6*, 331–340. [CrossRef]
19. Beer, G. Algorithms for geometrical operations with NURBS surfaces. *arXiv* **2022**, arXiv:2210.13160.
20. William, M.; Elaine Cohen, R.F.; Shirley, P. Practical Ray Tracing of Trimmed NURBS Surfaces. *J. Graph. Tools* **2000**, *5*, 27–52. [CrossRef]
21. Laurent, B. Implicit matrix representations of rational Bézier curves and surfaces. *Comput.-Aided Des.* **2014**, *46*, 14–24. [CrossRef]
22. Shen, J.; Busé, L.; Alliez, P.; Dodgson, N. A line/trimmed NURBS surface intersection algorithm using matrix representations. *Comput. Aided Geom. Des.* **2016**, *48*, 1–16. [CrossRef]
23. Wang, X.; Zhang, W.; Huang, X. Computation of point inversion and ray-surface intersection through tracing along the base surface. *Vis. Comput.* **2015**, *31*, 1487–1500. [CrossRef]
24. Feng, C.; Liang, J.; Ren, M.; Qiao, G.; Lu, W.; Liu, S. A fast hole-filling method for triangular mesh in additive repair. *Appl. Sci.* **2020**, *10*, 969. [CrossRef]
25. Park, J.H.; Park, S.; Yoon, S.H. Parametric blending of hole patches based on shape difference. *Symmetry* **2020**, *12*, 1759. [CrossRef]
26. Jun, Y. A piecewise hole filling algorithm in reverse engineering. *Comput.-Aided Des.* **2005**, *37*, 263–270. [CrossRef]
27. Zhao, W.; Gao, S.; Lin, H. A robust hole-filling algorithm for triangular mesh. *Vis. Comput.* **2007**, *23*, 987–997. [CrossRef]
28. Gaur, P.K.; Bose, S.K. On recent advances in 2D constrained Delaunay triangulation algorithms. *arXiv* **2017**, arXiv:1707.05949.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.